Knowledge Library

Knowledge Library

Purpose: provide **approved reference material** that improves quality and consistency **without becoming a code-donor**.

What it is (in AXION terms)

- A curated set of **reference units** ("knowledge items") that agents can consult.
- Each item has metadata that controls:
  - **when it's applicable**
  - **how it may be used** (pattern-only vs reusable)
  - **what must be cited/logged**
  - **whether it is safe for external agents**

What the Knowledge Library must include

1) Knowledge Item Types (what content goes in)

These are the categories that actually help template creation:

A) Patterns and playbooks (pattern-only)

- Architecture patterns (auth, payments, realtime, file uploads)
- Error handling patterns
- Logging/observability patterns
- API design patterns
- Data modeling patterns

Use rule: guidance only; do not copy code blocks unless allowlisted.

B) Examples (restricted by default)

- example template fills
- example specs
- example outputs (render envelopes, selection results)
- example gate reports

Use rule: may be used for structure; copying is blocked unless explicitly allowlisted.

C) Checklists

- security checklist per domain
- compliance checklist (PII, secrets, retention)
- UI/UX checklists (a11y, navigation constraints)
- testing checklists

Use rule: can be freely reused because it's "rules," not expressive content.

D) Definitions / glossary extensions

- terms, aliases, naming conventions
- "what does X mean in AXION"

Use rule: canonical definitions; must match Terms Registry.

E) Test vectors / known pitfalls

- "common failure modes" and how to avoid them
- "gotchas" for tools/frameworks

Use rule: safe guidance.

---

2) Knowledge Index (the registry for the library)

You need a **KNOWLEDGE_INDEX** parallel to your other registries.

Each knowledge item should have:

- `kid` (knowledge id)
- `title`
- `type` (pattern/checklist/example/definition/pitfall)
- `scope_tags` (auth, payments, realtime, mobile, etc.)
- `applicability` (mode/profile/executor type)
- `use_policy`
    - `pattern_only | reusable_with_allowlist | forbidden_for_external`
- `source` (internal authored vs external licensed)
- `license` (if external)
- `allowed_excerpt_limit` (text/code)
- `required_attribution` (if any)
- `redaction_class` (public/internal/restricted)

This is what prevents the library from becoming “random docs.”

---

3) How agents are allowed to use it (core behavior rules)

This is the “study guide” behavior you want.

Allowed

- Learn patterns and constraints
- Translate knowledge into *original* outputs
- Reference checklists and definitions
- Use examples to understand **structure**, not to copy content

Disallowed by default

- Copy code blocks into templates or generated repos
- Copy long prose verbatim into outputs
- Use restricted items when executor is external

Required behavior

- When an agent uses a knowledge item, it must write:
    - `knowledge_citations` into its output envelope OR into a run log
    - (if reuse occurs) `reuse_log` entry

---

4) Knowledge selection (how items get “called”)

You don’t want the agent rummaging.

You want a deterministic selection step:

- input: canonical spec + standards snapshot + run profile + executor type
- output: “recommended knowledge set” (KIDs)

This can be:

- a stage (KSL-SELECT)
- or part of template selection logic

Either way, the selection output should be saved as an artifact:

- `knowledge_selection.json`

---

5) Gates for Knowledge Library safety

Since this library can cause IP issues, it needs enforcement:

- **GATE-KNOW-01**: knowledge items used must be referenced (citations exist)
- **GATE-KNOW-02**: external agents cannot access restricted knowledge items
- **GATE-KNOW-03**: no verbatim copying beyond allowed excerpt limits
- **GATE-KNOW-04**: reuse requires allowlist + reuse log entry

Knowledge Breakdown

Knowledge Library

Purpose: give agents a curated, governed reference system ("study guide") to improve template creation quality **without** turning into a copy-donor.

Core design rules

1. **Atomic items** (small units; no monolith docs)
2. **Indexed and tagged** (selection is deterministic)
3. **Use-policy governed** (pattern-only by default)
4. **Versioned** (reproducibility)
5. **Internal vs external access controlled**
6. **Citations are mandatory when used**

KL-1) Knowledge Item Model

A Knowledge Library entry is a **Knowledge Item** (KID).

KL-1.1 Knowledge Item types (allowed)

- `concept` (what/why)
- `pattern` (reusable approach; pattern-only)
- `procedure` (how-to steps)
- `checklist` (enforceable list)
- `reference` (tables, defaults, parameter lists)
- `pitfall` (common failure modes)

- `example` (restricted; structure learning only unless allowlisted)
- `glossary_term` (canonical definitions; ties into TERMS_REGISTRY)

KL-1.2 Knowledge Item required sections (content contract)

Every KID file must contain:

- **Summary** (1–3 lines)
- **When to use** (applicability)
- **Do / Don't** (guardrails)
- **Core content** (the knowledge)
- **Links** (optional: internal cross-links)
- **Proof / confidence** (tier + references)

KL-1.3 Knowledge Item metadata (enforcement contract)

Each item must include machine-readable metadata (frontmatter or JSON header):

- `kid`
- `title`
- `type`
- `pillar` (IT_END_TO_END | INDUSTRY_PLAYBOOKS | LANGUAGES_AND_LIBRARIES)
- `domains` / `subdomains`
- `tags`
- `maturity` (draft | reviewed | verified | golden)

- `use_policy` (pattern_only | reusable_with_allowlist | restricted_internal_only)
- `executor_access` (internal_only | internal_and_external)
- `license` (internal_owned | MIT | Apache-2.0 | etc.)
- `allowed_excerpt` (max words/lines if any)
- `supersedes` / `deprecated_by`
- `created_at`, `updated_at`, `owner`

KL-2) Knowledge Registries (Source of Truth)

These sit in `knowledge/INDEX/` per your file tree.

KL-2.1 KNOWLEDGE_INDEX (required)

Maps KID → metadata → file path.

- must be the authoritative lookup for:
    - what exists
    - where it lives
    - allowed use policy
    - maturity
    - access constraints

KL-2.2 TAXONOMY (required)

Canonical taxonomy for:

- pillars
- domains/subdomains
- industries
- stacks/framework families
- tag vocabulary and aliases

KL-2.3 BUNDLES_INDEX (optional but recommended)

Curated sets of KIDs for common contexts:

- by run profile (API/Web/Mobile/etc.)
- by risk class (prototype/production/compliance)
- by executor (internal/external)

KL-2.4 SOURCES_INDEX (required if you ingest external content)

Tracks provenance/licensing:

- source type (internal authored, public domain, OSS docs)
- license
- attribution requirements
- allowed reuse level

KL-2.5 DEPRECATIONS + CHANGELOG (required)

- deprecated KIDs → replacements
- library-level change log

KL-3) Knowledge Selection (How agents “use it” safely)

Agents should not roam. The system selects.

KL-3.1 Inputs to selection

- canonical spec signals (domain/feature requirements)
- standards snapshot
- run profile (API/Web/etc.)
- risk class
- executor type (internal vs external)
- target stack (if known)

KL-3.2 Output: KNOWLEDGE_SELECTION artifact (required)

A deterministic artifact like:

- selected KIDs (ordered)
- why each KID was selected (tags/applicability match)
- restrictions attached (use_policy, excerpt limits)
- bundle IDs used (if any)

KL-3.3 Selection rules (minimum)

- filter by executor access
- filter by applicability
- filter by maturity threshold (e.g., production requires reviewed+)

- rank by relevance (tags + domain match)
- cap selection set (avoid overload)

KL-4) Usage Rules (what agents may do)

This is where your plagiarism/IP policy hooks in.

KL-4.1 Default rule

- Knowledge items are **pattern-only** unless explicitly allowlisted.

KL-4.2 Allowed behaviors

- derive requirements and constraints
- follow procedures and checklists
- re-express concepts in original language
- use examples to understand format (not copy)

KL-4.3 Restricted behaviors

- copying code blocks or long prose unless `reusable_with_allowlist`
- using internal-only items with external agents
- including restricted content inside kits for external execution

KL-4.4 Mandatory citation behavior

When a KID is used, agent must emit:

- `knowledge_citations: [KID-...]` in render envelope / run log

- if reuse: update reuse_log

KL-5) Enforcement Gates (Knowledge safety and integrity)

These make the library “real” and safe.

KL-5.1 Integrity gates

- **KL-GATE-01**: every referenced KID exists in KNOWLEDGE_INDEX
- **KL-GATE-02**: all KID metadata valid (type, tags, policies)
- **KL-GATE-03**: no deprecated KIDs used unless explicitly allowed

KL-5.2 Access gates

- **KL-GATE-04**: external executor cannot access restricted KIDs
- **KL-GATE-05**: kit export for external use excludes restricted content

KL-5.3 Plagiarism/reuse gates

- **KL-GATE-06**: if output contains allowlisted excerpts, reuse_log must include entry
- **KL-GATE-07**: block verbatim copying beyond excerpt limits

KL-5.4 Quality gates (optional but strong)

- **KL-GATE-08**: production runs require maturity >= reviewed for selected KIDs

---

KL-6) Scaling Over Time (content ops)

This is how it grows without rotting.

KL-6.1 Growth model

- add items atomically
- update taxonomy conservatively
- use bundles to create curated “slices”

KL-6.2 Maturity workflow

- draft → reviewed → verified → golden
- reviewed means: checked for correctness + policy alignment
- verified means: validated against real runs or tests
- golden means: stable canonical reference; changes are controlled

KL-6.3 Deprecation workflow

- mark deprecated
- add replacement KID
- keep aliases for a window
- block new selection of deprecated KIDs (except repro)

KL-6.4 External content ingestion rules

If you ingest outside material:

- store source metadata
- do not store copyrighted text verbatim unless licensed

- summarize + cite; keep excerpts within allowed limits
- enforce through `SOURCES_INDEX` + use_policy gates

KL-7) Minimum Viable Knowledge Library (what you should build first)

Given your “everything IT + industries + languages” goal, don’t start with “everything.” Start with a scalable spine:

1. **Taxonomy + index system**
2. **IT_END_TO_END**: security, networking, databases, auth, CI/CD, observability (core)
3. **Languages**: TypeScript/Node, React, SQL/Postgres (core)
4. **Industry playbooks**: 3–5 industries to start (healthcare, finance, ecommerce, logistics, gov)

Then scale by adding atomic KIDs and bundles.

KL-1) Knowledge Item Model

KL-1) Knowledge Item Model

A Knowledge Library entry is a **Knowledge Item** (KID).

KL-1.1 Knowledge Item types (allowed)

- `concept` (what/why)
- `pattern` (reusable approach; pattern-only)
- `procedure` (how-to steps)
- `checklist` (enforceable list)
- `reference` (tables, defaults, parameter lists)
- `pitfall` (common failure modes)
- `example` (restricted; structure learning only unless allowlisted)
- `glossary_term` (canonical definitions; ties into TERMS_REGISTRY)

KL-1.2 Knowledge Item required sections (content contract)

Every KID file must contain:

- **Summary** (1–3 lines)
- **When to use** (applicability)
- **Do / Don't** (guardrails)
- **Core content** (the knowledge)
- **Links** (optional: internal cross-links)
- **Proof / confidence** (tier + references)

KL-1.3 Knowledge Item metadata (enforcement contract)

Each item must include machine-readable metadata (frontmatter or JSON header):

- `kid`
- `title`
- `type`
- `pillar` (IT_END_TO_END | INDUSTRY_PLAYBOOKS | LANGUAGES_AND_LIBRARIES)
- `domains` / `subdomains`
- `tags`
- `maturity` (draft | reviewed | verified | golden)
- `use_policy` (pattern_only | reusable_with_allowlist | restricted_internal_only)
- `executor_access` (internal_only | internal_and_external)
- `license` (internal_owned | MIT | Apache-2.0 | etc.)
- `allowed_excerpt` (max words/lines if any)
- `supersedes` / `deprecated_by`
- `created_at`, `updated_at`, `owner`

KL-1.1 Knowledge Item types

KL-1.1 — **Knowledge Item types (allowed)**

Piece 1: Type registry (authoritative)

File: `knowledge/contracts/KL-1.1_kid_types.json`

```
{
 "schema_version": "1.0.0",
 "contract_id": "KL-1.1",
 "name": "Knowledge Item Types",
 "allowed_types": [
   "concept",
   "pattern",
   "procedure",
   "checklist",
   "reference",
   "pitfall",
   "example",
   "glossary_term"
 ]
}
```

Piece 2: Type semantics (what each type MUST contain in Core Content)

File: `knowledge/contracts/KL-1.1_kid_type_semantics.md`

```
---
contract_id: KL-1.1a
schema_version: 1.0.0
applies_to: "KID Core content section"
enforcement_level: "hard"
---

# KL-1.1a — KID Type Semantics

This defines what the **Core content** section MUST look like per KID type.

## concept
Core content MUST include:
- Explanation of the concept in plain language
- Scope boundaries (what it covers / does not cover)
- At least 1 concrete example or scenario (not code)
```

pattern
Core content MUST include:
- Problem statement (what this pattern solves)
- Solution outline (reusable approach)
- Applicability conditions (when it is valid)
- Constraints/guardrails (what must be true)
- Implementation notes (non-copy, behavior-level)

procedure
Core content MUST include:
- Prerequisites
- Ordered steps (numbered)
- Expected outputs (what success looks like)
- Rollback or “safe stop” step (where applicable)

checklist
Core content MUST include:
- A checklist with testable items
- Each item must be phrased as a verifiable condition
- Optional: “Evidence” sub-bullets per item

reference
Core content MUST include at least one of:
- Table(s)
- Enum lists
- Defaults/limits
- Parameter definitions (name → meaning → allowed values)

No narrative longer than 8 lines in any subsection.

pitfall
Core content MUST include:
- Symptom(s)
- Root cause(s)
- Detection method(s)
- Fix / mitigation
- Prevention rule (so it doesn’t recur)

example
Core content MUST include:
- Minimal illustrative structure (not a full donor)
- “What to learn from this” bullets

Must NOT include:
- Large blocks of production code
- Copy-ready sections beyond excerpt limits

glossary_term
Core content MUST include:
- Canonical definition (1–4 lines)
- Context (where used in Axion)
- Synonyms/aliases (if any)
- Common confusion / non-examples (optional)

Piece 3: Type ↔ allowed default policies (deterministic defaults)

File: `knowledge/contracts/KL-1.1_type_policy_defaults.json`

```
{
 "schema_version": "1.0.0",
 "contract_id": "KL-1.1b",
 "defaults": {
   "concept": {
     "use_policy": "pattern_only",
     "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
   },
   "pattern": {
     "use_policy": "pattern_only",
     "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
   },
   "procedure": {
     "use_policy": "pattern_only",
     "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
   },
   "checklist": {
     "use_policy": "pattern_only",
     "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
   },
   "reference": {
     "use_policy": "pattern_only",
     "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
   },
   "pitfall": {
     "use_policy": "pattern_only",
     "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
   },
   "example": {
     "use_policy": "restricted_internal_only",
     "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
   },
```

```
    "glossary_term": {
      "use_policy": "pattern_only",
      "allowed_excerpt": { "max_words": 0, "max_lines": 0 }
    }
  }
}
```

KL-1.2 Knowledge Item required sections

KL-1.2 — Knowledge Item required sections (content contract)

Piece A — Required sections + strict order

File: `knowledge/contracts/KL-1.2_required_sections.md`

```
---
contract_id: KL-1.2
schema_version: 1.0.0
applies_to: "All KID markdown entries"
enforcement_level: "hard"
---

# KL-1.2 — Required Sections (Content Contract)

Every KID file MUST contain the following H2 sections, in this exact order:

1. ## Summary
2. ## When to use
3. ## Do / Don’t
4. ## Core content
5. ## Links
6. ## Proof / confidence

## Rules
- Section headings MUST match exactly (case + punctuation).
- "Links" MAY be empty, but the heading MUST exist.
- No extra required sections are allowed between these headings.
- Additional optional headings are allowed ONLY under "Core content" as H3/H4.
- A KID that fails section order or missing headings is invalid.
```

Piece B — Section-level constraints (length + formatting rules)

File: `knowledge/contracts/KL-1.2_section_constraints.md`

```
---
contract_id: KL-1.2a
schema_version: 1.0.0
applies_to: "KID section bodies"
enforcement_level: "hard"
```

KL-1.2a — Section Constraints

These rules keep items atomic and non-monolithic.

Summary
- MUST be 1–3 lines.
- MUST NOT contain:
 - code blocks
 - tables
 - links
- SHOULD use plain language and name the problem/benefit.

When to use
- MUST include at least 2 bullets OR a short paragraph + at least 2 bullets.
- MUST include applicability conditions (signals that selection can match).
- SHOULD include "When NOT to use" bullets when misuse risk is non-trivial.

Do / Don’t
- MUST include two sublists labeled **Do** and **Don’t**.
- Each sublist MUST have 2–8 bullets.
- Bullets MUST be concrete and testable where possible.
- MUST NOT include long prose paragraphs (max 6 lines in any single paragraph).

Core content
- MUST contain the knowledge content.
- May include subsections (H3/H4) but MUST remain atomic:
 - Soft cap: 600 words
 - Hard cap: 1,200 words (gate failure if exceeded)
- Code blocks:
 - Allowed only if item.type is pattern/procedure/reference/example
 - If use_policy is pattern_only, code blocks MUST be pseudocode or illustrative, not copy-ready production code
- Tables:
 - Allowed for reference items; allowed elsewhere only if they improve scanability.

Links
- Heading MUST exist (even if empty).
- Internal links SHOULD use KID IDs, not filesystem paths.
- External links allowed only if license + use_policy permit.

Proof / confidence
- MUST include:

- tier: draft | reviewed | verified | golden
- evidence: brief text (or "none yet" for draft)
- SHOULD include references (run IDs, tests, audits) when tier >= reviewed.

Piece C — Validation checklist (what the KL-1.2 gate checks)

File: `knowledge/contracts/KL-1.2_validation_checklist.md`

```
---
contract_id: KL-1.2b
schema_version: 1.0.0
applies_to: "KL validation gate for KID content sections"
enforcement_level: "hard"
---

# KL-1.2b — Validation Checklist

A KID passes KL-1.2 if ALL checks are true:

## Headings + order
- [ ] Contains H2 headings exactly: Summary, When to use, Do / Don't, Core content, Links, Proof / confidence
- [ ] Headings appear in the correct order with no missing required heading
- [ ] No additional H2 headings inserted between required headings

## Summary constraints
- [ ] Summary exists and is 1–3 lines
- [ ] Summary contains no links, code blocks, or tables

## When to use constraints
- [ ] When to use exists and includes applicability (>= 2 bullets OR paragraph + >=2 bullets)

## Do / Don't constraints
- [ ] Do / Don't contains both labeled sublists: Do and Don't
- [ ] Do list has 2–8 bullets
- [ ] Don't list has 2–8 bullets

## Core content constraints
- [ ] Core content exists
- [ ] Word count <= 1,200 (hard cap)
- [ ] If code blocks exist:
  - [ ] type in {pattern, procedure, reference, example}
```

- [ ] if use_policy == pattern_only: blocks are illustrative/pseudocode (no copy-ready donor chunks)

Links constraints
- [ ] Links heading exists (content may be empty)

Proof / confidence constraints
- [ ] Proof / confidence exists
- [ ] Contains tier: {draft|reviewed|verified|golden}
- [ ] Contains evidence line (or “none yet” if tier==draft)

KL-1.3 Knowledge Item metadata

KL-1.3 — Knowledge Item metadata (enforcement contract)

Below is **KL-1.3** with **all pieces** (complete coverage).

Piece A — Canonical metadata schema (JSON Schema)

File: `knowledge/contracts/KL-1.3_kid_metadata.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "contract_id": "KL-1.3",
 "title": "KID Metadata (Frontmatter/JSON Header)",
 "type": "object",
 "required": [
   "kid",
   "title",
   "type",
   "pillar",
   "domains",
   "tags",
   "maturity",
   "use_policy",
   "executor_access",
   "license",
   "allowed_excerpt",
   "created_at",
   "updated_at",
   "owner"
 ],
 "properties": {
   "kid": {
     "type": "string",
     "description": "Stable Knowledge Item ID",
     "pattern": "^KID-[A-Z0-9_]+-[A-Z0-9_]+-[A-Z0-9_]+-\\d{4,}$"
   },
   "title": { "type": "string", "minLength": 3 },

   "type": {
```

```
      "type": "string",
      "enum": [
        "concept",
        "pattern",
        "procedure",
        "checklist",
        "reference",
        "pitfall",
        "example",
        "glossary_term"
      ]
    },

    "pillar": {
      "type": "string",
      "enum": ["IT_END_TO_END", "INDUSTRY_PLAYBOOKS",
"LANGUAGES_AND_LIBRARIES"]
    },

    "domains": {
      "type": "array",
      "minItems": 1,
      "items": { "type": "string", "minLength": 2 }
    },

    "subdomains": {
      "type": "array",
      "items": { "type": "string", "minLength": 2 }
    },

    "tags": {
      "type": "array",
      "minItems": 1,
      "items": { "type": "string", "minLength": 2 }
    },

    "maturity": {
      "type": "string",
      "enum": ["draft", "reviewed", "verified", "golden"]
    },

    "use_policy": {
      "type": "string",
      "enum": ["pattern_only", "reusable_with_allowlist", "restricted_internal_only"]
```

```
  },

  "executor_access": {
    "type": "string",
    "enum": ["internal_only", "internal_and_external"]
  },

  "license": { "type": "string", "minLength": 2 },

  "allowed_excerpt": {
    "type": "object",
    "required": ["max_words", "max_lines"],
    "properties": {
      "max_words": { "type": "integer", "minimum": 0 },
      "max_lines": { "type": "integer", "minimum": 0 }
    },
    "additionalProperties": false
  },

  "supersedes": {
    "type": "string",
    "description": "KID this item replaces (optional)"
  },

  "deprecated_by": {
    "type": "string",
    "description": "KID that replaces this item (optional)"
  },

  "created_at": { "type": "string", "format": "date" },
  "updated_at": { "type": "string", "format": "date" },
  "owner": { "type": "string", "minLength": 2 }
 },
 "additionalProperties": false
}
```

Piece B — Metadata normalization rules (deterministic)

File: `knowledge/contracts/KL-1.3_metadata_normalization.md`

```
---
contract_id: KL-1.3a
schema_version: 1.0.0
```

applies_to: "KID metadata fields"
enforcement_level: "hard"

KL-1.3a — Metadata Normalization Rules

IDs
- kid MUST be immutable once created.
- kid MUST match filename (without .md).
- kid MUST match the ID pattern: KID-<PILLAR>-<DOMAIN>-<TYPE>-<NNNN>

Domains / subdomains
- domains MUST use canonical domain slugs from TAXONOMY.json.
- subdomains (if present) MUST use canonical subdomain slugs from TAXONOMY.json.
- If multiple domains are provided, the first is treated as the primary domain.

Tags
- tags MUST use canonical tag vocabulary from TAXONOMY.json, including aliases resolution.
- tags MUST be lowercase snake_case.
- tags MUST be de-duplicated and sorted lexicographically.

Maturity
- maturity transitions allowed:
 - draft → reviewed → verified → golden
- reverse transitions require DEPRECATIONS entry + changelog note.

use_policy defaults
- Default use_policy is pattern_only unless explicitly changed.
- example types SHOULD default to restricted_internal_only.

executor_access
- internal_only MUST be enforced for external executors during selection (KL-3/KL-5).

allowed_excerpt
- max_words/max_lines default to 0/0.
- If either value > 0 then use_policy MUST be reusable_with_allowlist.

supersedes / deprecated_by
- If deprecated_by is set:
 - this KID MUST appear in DEPRECATIONS.json mapping to replacement
 - selection must avoid it unless repro mode is enabled (KL-5)

Piece C — Frontmatter template (canonical)

File: `knowledge/templates/KID_frontmatter_template.md`

```
---
kid: KID-<PILLAR>-<DOMAIN>-<TYPE>-0001
title: "<Title>"
type: concept | pattern | procedure | checklist | reference | pitfall | example | glossary_term
pillar: IT_END_TO_END | INDUSTRY_PLAYBOOKS | LANGUAGES_AND_LIBRARIES
domains: ["<domain_slug>"]
subdomains: []
tags: ["<tag1>", "<tag2>"]
maturity: draft | reviewed | verified | golden
use_policy: pattern_only | reusable_with_allowlist | restricted_internal_only
executor_access: internal_only | internal_and_external
license: internal_owned | MIT | Apache-2.0 | etc.
allowed_excerpt:
 max_words: 0
 max_lines: 0
supersedes: ""
deprecated_by: ""
created_at: 2026-03-05
updated_at: 2026-03-05
owner: axion
---

# <Title>

## Summary
## When to use
## Do / Don't
## Core content
## Links
## Proof / confidence
```

Piece D — Metadata validation checklist (gate view)

File: `knowledge/contracts/KL-1.3_validation_checklist.md`

```
---
contract_id: KL-1.3b
schema_version: 1.0.0
applies_to: "KL validation gate for KID metadata"
enforcement_level: "hard"
---
```

KL-1.3b — Metadata Validation Checklist

A KID passes KL-1.3 if ALL checks are true:

Presence + types
- [ ] All required fields exist (kid, title, type, pillar, domains, tags, maturity, use_policy, executor_access, license, allowed_excerpt, created_at, updated_at, owner)
- [ ] allowed_excerpt contains max_words and max_lines (ints >= 0)

ID integrity
- [ ] kid matches the required pattern
- [ ] filename == kid + ".md"

Canonical vocab
- [ ] pillar is valid enum
- [ ] type is valid enum
- [ ] domains/subdomains exist in TAXONOMY (if taxonomy enforcement enabled)
- [ ] tags exist in TAXONOMY vocabulary (or alias-resolved)

Policy coherence
- [ ] if allowed_excerpt.max_words > 0 OR max_lines > 0 → use_policy == reusable_with_allowlist
- [ ] if type == example → use_policy should be restricted_internal_only unless explicitly overridden

Deprecation coherence
- [ ] if deprecated_by is set → entry exists in DEPRECATIONS.json mapping this KID → deprecated_by

KL-2) Knowledge Registries

KL-2) Knowledge Registries (Source of Truth)

These sit in `knowledge/INDEX/` per your file tree.

KL-2.1 KNOWLEDGE_INDEX (required)

Maps KID → metadata → file path.

- must be the authoritative lookup for:
    - what exists
    - where it lives
    - allowed use policy
    - maturity
    - access constraints

KL-2.2 TAXONOMY (required)

Canonical taxonomy for:

- pillars
- domains/subdomains
- industries
- stacks/framework families
- tag vocabulary and aliases

KL-2.3 BUNDLES_INDEX (optional but recommended)

Curated sets of KIDs for common contexts:

- by run profile (API/Web/Mobile/etc.)
- by risk class (prototype/production/compliance)
- by executor (internal/external)

KL-2.4 SOURCES_INDEX (required if you ingest external content)

Tracks provenance/licensing:

- source type (internal authored, public domain, OSS docs)
- license
- attribution requirements
- allowed reuse level

KL-2.5 DEPRECATIONS + CHANGELOG (required)

- deprecated KIDs → replacements
- library-level change log

KL-2.1 KNOWLEDGE_INDEX

KL-2.1 — KNOWLEDGE_INDEX (required)

Piece A — Registry schema (JSON Schema)

File: `knowledge/contracts/KL-2.1_knowledge_index.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "contract_id": "KL-2.1",
 "title": "KNOWLEDGE_INDEX Registry Schema",
 "type": "object",
 "required": ["schema_version", "items"],
 "properties": {
   "schema_version": { "type": "string", "minLength": 1 },
   "items": {
     "type": "array",
     "items": { "$ref": "#/$defs/index_item" }
   }
 },
 "$defs": {
   "index_item": {
     "type": "object",
     "required": [
       "kid",
       "title",
       "type",
       "pillar",
       "domains",
       "tags",
       "maturity",
       "use_policy",
       "executor_access",
       "license",
       "allowed_excerpt",
       "path",
       "created_at",
       "updated_at",
       "owner"
     ],
     "properties": {
       "kid": { "type": "string", "minLength": 8 },
```

"title": { "type": "string", "minLength": 3 },
"type": {
"type": "string",
"enum": [
"concept",
"pattern",
"procedure",
"checklist",
"reference",
"pitfall",
"example",
"glossary_term"
]
},
"pillar": {
"type": "string",
"enum": ["IT_END_TO_END", "INDUSTRY_PLAYBOOKS",
"LANGUAGES_AND_LIBRARIES"]
},
"domains": {
"type": "array",
"minItems": 1,
"items": { "type": "string", "minLength": 2 }
},
"subdomains": {
"type": "array",
"items": { "type": "string", "minLength": 2 }
},
"tags": {
"type": "array",
"minItems": 1,
"items": { "type": "string", "minLength": 2 }
},
"maturity": {
"type": "string",
"enum": ["draft", "reviewed", "verified", "golden"]
},
"use_policy": {
"type": "string",
"enum": ["pattern_only", "reusable_with_allowlist", "restricted_internal_only"]
},
"executor_access": {
"type": "string",
"enum": ["internal_only", "internal_and_external"]

```
      },
      "license": { "type": "string", "minLength": 2 },
      "allowed_excerpt": {
        "type": "object",
        "required": ["max_words", "max_lines"],
        "properties": {
          "max_words": { "type": "integer", "minimum": 0 },
          "max_lines": { "type": "integer", "minimum": 0 }
        },
        "additionalProperties": false
      },
      "path": { "type": "string", "minLength": 5 },
      "supersedes": { "type": "string" },
      "deprecated_by": { "type": "string" },
      "created_at": { "type": "string", "format": "date" },
      "updated_at": { "type": "string", "format": "date" },
      "owner": { "type": "string", "minLength": 2 }
    },
    "additionalProperties": false
  }
},
"additionalProperties": false
}
```

Piece B — Authoritative rules (what KNOWLEDGE_INDEX governs)

File: `knowledge/contracts/KL-2.1_knowledge_index_rules.md`

```
---
contract_id: KL-2.1a
schema_version: 1.0.0
applies_to: "knowledge/INDEX/KNOWLEDGE_INDEX.json"
enforcement_level: "hard"
---

# KL-2.1a — KNOWLEDGE_INDEX Rules

## Purpose
KNOWLEDGE_INDEX is the authoritative lookup for:
- what KIDs exist
- where each KID lives (path)
- use_policy + excerpt limits
- maturity tier
- executor access constraints
```

Authority & conflict resolution
- If KNOWLEDGE_INDEX conflicts with KID frontmatter:
 - This is a gate failure.
 - Default resolution is: update KNOWLEDGE_INDEX to match the KID frontmatter (frontmatter is source-of-truth for metadata).
 - Exception: if the KID file is missing/invalid, the index entry must be removed or corrected.

Uniqueness constraints
- kid MUST be unique across all items
- path MUST be unique across all items

Sorting (determinism)
When emitting a selection list or any derived output:
Sort items by:
1) pillar
2) primary domain (domains[0])
3) type
4) kid

Update discipline
- Adding a KID requires:
 - KID file exists
 - KID metadata passes KL-1.3
 - KID sections pass KL-1.2
 - Index entry added/updated
- Removing a KID requires:
 - Deprecation mapping OR explicit delete recorded in CHANGELOG

Piece C — Minimal blank registry file (scaffold)

File: `knowledge/INDEX/KNOWLEDGE_INDEX.json`

```
{
 "schema_version": "1.0.0",
 "items": []
}
```

Piece D — Validation checklist (gate view)

File: `knowledge/contracts/KL-2.1_validation_checklist.md`

---
contract_id: KL-2.1b

schema_version: 1.0.0
applies_to: "KL integrity validation for KNOWLEDGE_INDEX"
enforcement_level: "hard"

KL-2.1b — KNOWLEDGE_INDEX Validation Checklist

Registry passes if:

Schema
- [ ] JSON parses
- [ ] Matches KL-2.1 schema

Uniqueness
- [ ] No duplicate kid
- [ ] No duplicate path

Referential integrity (optional strict mode)
- [ ] For every item.path, file exists
- [ ] KID filename matches item.kid
- [ ] KID frontmatter kid matches item.kid
- [ ] KID frontmatter fields match index fields for:
title/type/pillar/domains/tags/maturity/use_policy/executor_access/license/allowed_excerpt

Deprecations coherence
- [ ] If item.deprecated_by exists, DEPRECATIONS.json contains mapping item.kid ->
item.deprecated_by

KL-2.2 TAXONOMY

KL-2.2 — TAXONOMY (required)

Piece A — TAXONOMY schema (JSON Schema)

File: `knowledge/contracts/KL-2.2_taxonomy.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "contract_id": "KL-2.2",
 "title": "TAXONOMY Registry Schema",
 "type": "object",
 "required": [
   "schema_version",
   "pillars",
   "domains",
   "tag_vocabulary",
   "tag_aliases",
   "industries",
   "stack_families"
 ],
 "properties": {
   "schema_version": { "type": "string", "minLength": 1 },

   "pillars": {
     "type": "array",
     "minItems": 1,
     "items": {
       "type": "string",
       "enum": ["IT_END_TO_END", "INDUSTRY_PLAYBOOKS",
"LANGUAGES_AND_LIBRARIES"]
     }
   },

   "domains": {
     "type": "object",
     "description": "Canonical domain registry keyed by domain_slug",
     "additionalProperties": { "$ref": "#/$defs/domain" }
   },

   "tag_vocabulary": {
     "type": "object",
```

```
    "description": "Canonical tags keyed by tag",
    "additionalProperties": { "$ref": "#/$defs/tag" }
  },

  "tag_aliases": {
    "type": "object",
    "description": "alias -> canonical_tag mapping",
    "additionalProperties": { "type": "string", "minLength": 2 }
  },

  "industries": {
    "type": "object",
    "description": "Industry playbooks registry keyed by industry_slug",
    "additionalProperties": { "$ref": "#/$defs/industry" }
  },

  "stack_families": {
    "type": "object",
    "description": "Framework families keyed by stack_slug",
    "additionalProperties": { "$ref": "#/$defs/stack_family" }
  }
},

"$defs": {
  "domain": {
    "type": "object",
    "required": ["name", "pillar", "subdomains", "min_tags"],
    "properties": {
      "name": { "type": "string", "minLength": 2 },
      "pillar": {
        "type": "string",
        "enum": ["IT_END_TO_END", "INDUSTRY_PLAYBOOKS",
"LANGUAGES_AND_LIBRARIES"]
      },
      "subdomains": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/subdomain" }
      },
      "min_tags": {
        "type": "array",
        "minItems": 0,
        "items": { "type": "string", "minLength": 2 }
      }
    },
```

```
  "additionalProperties": false
},

"subdomain": {
  "type": "object",
  "required": ["name"],
  "properties": {
    "name": { "type": "string", "minLength": 2 }
  },
  "additionalProperties": false
},

"tag": {
  "type": "object",
  "required": ["name", "category"],
  "properties": {
    "name": { "type": "string", "minLength": 2 },
    "category": {
      "type": "string",
      "enum": [
        "security",
        "architecture",
        "data",
        "infra",
        "testing",
        "performance",
        "observability",
        "product",
        "language",
        "framework",
        "industry",
        "general"
      ]
    },
    "notes": { "type": "string" }
  },
  "additionalProperties": false
},

"industry": {
  "type": "object",
  "required": ["name", "regulated"],
  "properties": {
    "name": { "type": "string", "minLength": 2 },
```

```
      "regulated": { "type": "boolean" },
      "notes": { "type": "string" }
    },
    "additionalProperties": false
  },

  "stack_family": {
    "type": "object",
    "required": ["name", "ecosystem"],
    "properties": {
      "name": { "type": "string", "minLength": 2 },
      "ecosystem": { "type": "string", "minLength": 2 },
      "notes": { "type": "string" }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece B — TAXONOMY rules (canonical slugs, aliasing, conservative updates)

File: `knowledge/contracts/KL-2.2_taxonomy_rules.md`

```
---
contract_id: KL-2.2a
schema_version: 1.0.0
applies_to: "knowledge/INDEX/TAXONOMY.json"
enforcement_level: "hard"
---

# KL-2.2a — TAXONOMY Rules

## Purpose
TAXONOMY is the canonical vocabulary for:
- pillars
- domains/subdomains
- industries
- stack/framework families
- tag vocabulary + aliases
```

Slug rules
- domain_slug, subdomain_slug, industry_slug, stack_slug:
 - lowercase snake_case
 - stable once created (do not rename; deprecate instead)
- tag keys MUST be lowercase snake_case.

Tag aliasing
- tag_aliases maps non-canonical labels to canonical tags.
- On ingest:
 - if tag is canonical -> keep
 - else if tag is an alias -> replace with canonical
 - else -> reject (or require TAXONOMY update) depending on strictness mode

Conservative updates
- Adding tags/domains is allowed.
- Removing or renaming requires:
 - entry in DEPRECATIONS.json (for tags/domains)
 - changelog entry
 - a migration plan for existing KIDs/index entries

Minimum domain tags (min_tags)
- Each domain defines a minimum tag set used for quality:
 - if a KID is in that domain, it SHOULD include at least 1 tag from min_tags.
 - production selection may require this (optional gate).

Piece C — Minimal scaffold file (blank but valid)

File: `knowledge/INDEX/TAXONOMY.json`

```
{
 "schema_version": "1.0.0",
 "pillars": ["IT_END_TO_END", "INDUSTRY_PLAYBOOKS",
"LANGUAGES_AND_LIBRARIES"],
 "domains": {},
 "tag_vocabulary": {},
 "tag_aliases": {},
 "industries": {},
 "stack_families": {}
}
```

Piece D — Seed example (optional, but shows intended shape)

File: `knowledge/INDEX/TAXONOMY.seed.example.json`

```
{
 "schema_version": "1.0.0",
 "pillars": ["IT_END_TO_END", "INDUSTRY_PLAYBOOKS",
"LANGUAGES_AND_LIBRARIES"],
 "domains": {
   "security_fundamentals": {
     "name": "Security Fundamentals",
     "pillar": "IT_END_TO_END",
     "subdomains": {
       "auth": { "name": "Authentication" },
       "authz": { "name": "Authorization" },
       "secrets": { "name": "Secrets Management" }
     },
     "min_tags": ["auth", "authz", "secrets", "threat_model"]
   }
 },
 "tag_vocabulary": {
   "auth": { "name": "Authentication", "category": "security" },
   "authz": { "name": "Authorization", "category": "security" },
   "secrets": { "name": "Secrets", "category": "security" },
   "threat_model": { "name": "Threat Modeling", "category": "security" }
 },
 "tag_aliases": {
   "authentication": "auth",
   "authorization": "authz",
   "secret_management": "secrets",
   "threatmodel": "threat_model"
 },
 "industries": {
   "healthcare": { "name": "Healthcare", "regulated": true, "notes": "HIPAA risk context." }
 },
 "stack_families": {
   "react": { "name": "React", "ecosystem": "frontend", "notes": "React + tooling families." }
 }
}
```

Piece E — Validation checklist (gate view)

File: `knowledge/contracts/KL-2.2_validation_checklist.md`

---

contract_id: KL-2.2b
schema_version: 1.0.0
applies_to: "KL integrity validation for TAXONOMY"
enforcement_level: "hard"

KL-2.2b — TAXONOMY Validation Checklist

Passes if:

Schema
- [ ] JSON parses
- [ ] Matches KL-2.2 schema

Slugs
- [ ] All keys (domain/tag/industry/stack) are lowercase snake_case

Domain integrity
- [ ] Each domain has pillar, subdomains, min_tags
- [ ] Each subdomain has a name

Tag integrity
- [ ] Every canonical tag in tag_vocabulary has name + category
- [ ] Every alias in tag_aliases maps to an existing canonical tag

Optional strict referential checks (recommended)
- [ ] Every KID in KNOWLEDGE_INDEX uses only domains/tags present in TAXONOMY (after alias normalization)

KL-2.3 BUNDLES_INDEX

KL-2.3 — BUNDLES_INDEX (optional but recommended)

Piece A — Bundle registry schema (JSON Schema)

File: `knowledge/contracts/KL-2.3_bundles_index.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "contract_id": "KL-2.3",
 "title": "BUNDLES_INDEX Registry Schema",
 "type": "object",
 "required": ["schema_version", "bundles"],
 "properties": {
   "schema_version": { "type": "string", "minLength": 1 },
   "bundles": {
     "type": "array",
     "items": { "$ref": "#/$defs/bundle" }
   }
 },
 "$defs": {
   "bundle": {
     "type": "object",
     "required": [
       "bundle_id",
       "title",
       "description",
       "selection_policy",
       "kids",
       "created_at",
       "updated_at",
       "owner"
     ],
     "properties": {
       "bundle_id": {
         "type": "string",
         "pattern": "^BUNDLE-[A-Z0-9_]+-\\d{4,}$"
       },
       "title": { "type": "string", "minLength": 3 },
       "description": { "type": "string", "minLength": 3 },

       "by_run_profile": {
```

```
  "type": "array",
  "items": { "type": "string" },
  "description": "Optional: contexts like API, WEB, MOBILE, INFRA"
},

"by_risk_class": {
  "type": "array",
  "items": { "type": "string" },
  "description": "Optional: PROTOTYPE, PROD, COMPLIANCE, etc."
},

"by_executor": {
  "type": "array",
  "items": {
    "type": "string",
    "enum": ["internal_only", "internal_and_external"]
  }
},

"by_stack_family": {
  "type": "array",
  "items": { "type": "string" }
},

"domains": {
  "type": "array",
  "items": { "type": "string" }
},

"tags": {
  "type": "array",
  "items": { "type": "string" }
},

"kids": {
  "type": "array",
  "minItems": 1,
  "items": { "type": "string" }
},

"selection_policy": {
  "type": "object",
  "required": ["mode", "cap"],
  "properties": {
```

```
          "mode": {
            "type": "string",
            "enum": ["append", "override", "scoped"]
          },
          "cap": { "type": "integer", "minimum": 1 },
          "min_maturity": {
            "type": "string",
            "enum": ["draft", "reviewed", "verified", "golden"]
          }
        },
        "additionalProperties": false
      },

      "deprecated_by": { "type": "string" },

      "created_at": { "type": "string", "format": "date" },
      "updated_at": { "type": "string", "format": "date" },
      "owner": { "type": "string", "minLength": 2 }
    },
    "additionalProperties": false
  }
 },
 "additionalProperties": false
}
```

Piece B — Bundle rules (how they're used safely)

File: `knowledge/contracts/KL-2.3_bundle_rules.md`

```
---
contract_id: KL-2.3a
schema_version: 1.0.0
applies_to: "knowledge/INDEX/BUNDLES_INDEX.json"
enforcement_level: "hard"
---

# KL-2.3a — Bundle Rules

## Purpose
Bundles are curated sets of KIDs used to create deterministic "slices" of the library.

## What a bundle can do
- Provide a starter set for a run profile (API/Web/Mobile/etc.)
```

- Provide a starter set for a risk class (prototype vs production vs compliance)
- Provide a starter set for an executor (internal vs external)
- Provide a starter set for a target stack family (React/Node/Postgres/etc.)

Determinism
- bundle_id is immutable.
- The order of "kids" is meaningful and MUST be preserved during selection.
- Selection artifact MUST record:
 - bundle_ids used
 - the resolved ordered KID list after applying bundle selection_policy

Policy safety
- Bundles MUST NOT bypass KID-level rules:
 - executor_access filtering still applies
 - maturity thresholds still apply
 - deprecated avoidance still applies

Deprecation
- If bundle is replaced, use deprecated_by and record mapping in DEPRECATIONS.json (bundle-level section).

Piece C — Minimal scaffold file

File: `knowledge/INDEX/BUNDLES_INDEX.json`

```
{
 "schema_version": "1.0.0",
 "bundles": []
}
```

Piece D — Example bundle (starter slice)

File: `knowledge/INDEX/BUNDLES_INDEX.seed.example.json`

```
{
 "schema_version": "1.0.0",
 "bundles": [
   {
     "bundle_id": "BUNDLE-ITSEC_STARTER-0001",
     "title": "IT Security Fundamentals — Starter",
     "description": "Minimal baseline security KIDs for general builds.",
     "by_run_profile": ["API", "WEB", "MOBILE"],
```

```
    "by_risk_class": ["PROTOTYPE", "PROD"],
    "by_executor": ["internal_and_external"],
    "domains": ["security_fundamentals"],
    "tags": ["auth", "authz", "secrets", "threat_model"],
    "kids": [
      "KID-IT_END_TO_END-SECURITY_FUNDAMENTALS-CONCEPT-0001",
      "KID-IT_END_TO_END-SECURITY_FUNDAMENTALS-CHECKLIST-0001"
    ],
    "selection_policy": { "mode": "append", "cap": 25, "min_maturity": "draft" },
    "created_at": "2026-03-05",
    "updated_at": "2026-03-05",
    "owner": "axion"
  }
 ]
}
```


(Note: those KIDs are placeholders until you finalize your KID naming convention; the schema supports any stable ID string.)

Piece E — Validation checklist (gate view)

File: `knowledge/contracts/KL-2.3_validation_checklist.md`

```
---
contract_id: KL-2.3b
schema_version: 1.0.0
applies_to: "KL integrity validation for BUNDLES_INDEX"
enforcement_level: "hard"
---

# KL-2.3b — BUNDLES_INDEX Validation Checklist

Passes if:

## Schema
- [ ] JSON parses
- [ ] Matches KL-2.3 schema

## Determinism
- [ ] bundle_id is unique
- [ ] kids list has no duplicates within a bundle

## Optional strict referential checks (recommended)
```

- [ ] Every bundle.kids entry exists in KNOWLEDGE_INDEX
- [ ] Bundle does not include KIDs with executor_access that conflicts with bundle.by_executor
- [ ] Bundle min_maturity is not lower than the run’s required maturity (enforced during selection)

KL-2.4 SOURCES_INDEX

KL-2.4 — SOURCES_INDEX (required if you ingest external content)

Piece A — Sources registry schema (JSON Schema)

File: `knowledge/contracts/KL-2.4_sources_index.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "contract_id": "KL-2.4",
 "title": "SOURCES_INDEX Registry Schema",
 "type": "object",
 "required": ["schema_version", "sources"],
 "properties": {
   "schema_version": { "type": "string", "minLength": 1 },
   "sources": {
     "type": "array",
     "items": { "$ref": "#/$defs/source" }
   }
 },
 "$defs": {
   "source": {
     "type": "object",
     "required": [
       "source_id",
       "source_type",
       "title",
       "license",
       "attribution_required",
       "allowed_reuse_level",
       "created_at",
       "updated_at",
       "owner"
     ],
     "properties": {
       "source_id": {
         "type": "string",
         "pattern": "^SRC-[A-Z0-9_]+-\\d{4,}$"
       },
       "source_type": {
         "type": "string",
```

"enum": ["internal_authored", "public_domain", "oss_docs", "third_party_docs", "academic_paper"]
},
"title": { "type": "string", "minLength": 3 },

"origin": {
"type": "object",
"description": "Where the material comes from (no raw URLs required here; can store canonical identifiers).",
"properties": {
"publisher": { "type": "string" },
"author": { "type": "string" },
"year": { "type": "integer", "minimum": 0 },
"identifier": { "type": "string", "description": "Optional: doc id, ISBN, repo name, etc." }
},
"additionalProperties": false
},

"license": { "type": "string", "minLength": 2 },
"attribution_required": { "type": "boolean" },

"allowed_reuse_level": {
"type": "string",
"enum": [
"summary_only",
"short_excerpts_only",
"allowlisted_reuse",
"full_reuse_per_license"
]
},

"reuse_constraints": {
"type": "object",
"properties": {
"max_excerpt_words": { "type": "integer", "minimum": 0 },
"max_excerpt_lines": { "type": "integer", "minimum": 0 },
"notes": { "type": "string" }
},
"additionalProperties": false
},

"notes": { "type": "string" },

"created_at": { "type": "string", "format": "date" },

```
      "updated_at": { "type": "string", "format": "date" },
      "owner": { "type": "string", "minLength": 2 }
    },
    "additionalProperties": false
  }
},
"additionalProperties": false
}
```

Piece B — Provenance + licensing rules

File: `knowledge/contracts/KL-2.4_sources_rules.md`

```
---
contract_id: KL-2.4a
schema_version: 1.0.0
applies_to: "knowledge/INDEX/SOURCES_INDEX.json + any KID that references external
sources"
enforcement_level: "hard"
---

# KL-2.4a — Sources Rules (Provenance & Licensing)

## Purpose
SOURCES_INDEX governs provenance/licensing so the Knowledge Library does not become a
copy-donor.

## When SOURCES_INDEX is required
If ANY KID includes:
- text excerpts from external material
- code excerpts from external material
- direct reuse beyond paraphrase/summarization
then the KID MUST reference a source_id that exists in SOURCES_INDEX.

## allowed_reuse_level meanings
- summary_only:
 - no verbatim excerpts stored in KIDs
 - only paraphrase + citations
- short_excerpts_only:
 - excerpts allowed only within KID allowed_excerpt limits AND source constraints
- allowlisted_reuse:
 - reuse allowed only for explicitly allowlisted sections/assets
- full_reuse_per_license:
```

- reuse permitted as allowed by license (still must cite)

Consistency rules
- If a KID has allowed_excerpt > 0 or use_policy != pattern_only AND it is derived from external content:
  - it MUST include a source_id reference (in Links or in metadata extension field).
- If attribution_required is true:
  - KID Links must include attribution text (author/publisher/year) or a standardized attribution block.

External executor safety
- KIDs referencing sources with restricted reuse MUST NOT be exported in full to external kits unless allowed_reuse_level permits and excerpt limits are enforced (KL-5).

Piece C — Minimal scaffold file

File: `knowledge/INDEX/SOURCES_INDEX.json`

```
{
 "schema_version": "1.0.0",
 "sources": []
}
```

Piece D — Example source entries

File: `knowledge/INDEX/SOURCES_INDEX.seed.example.json`

```
{
 "schema_version": "1.0.0",
 "sources": [
   {
     "source_id": "SRC-OSS_DOCS-0001",
     "source_type": "oss_docs",
     "title": "Example OSS Documentation Source",
     "origin": { "publisher": "Example Project", "author": "Maintainers", "year": 2025, "identifier": "repo:example/project" },
     "license": "Apache-2.0",
     "attribution_required": true,
     "allowed_reuse_level": "short_excerpts_only",
     "reuse_constraints": { "max_excerpt_words": 50, "max_excerpt_lines": 5, "notes": "Excerpts only; prefer summaries." },
     "notes": "Used for reference-only KIDs; do not paste large blocks.",
```

"created_at": "2026-03-05",
"updated_at": "2026-03-05",
"owner": "axion"
}
]
}

Piece E — Validation checklist (gate view)

File: `knowledge/contracts/KL-2.4_validation_checklist.md`

---
contract_id: KL-2.4b
schema_version: 1.0.0
applies_to: "KL integrity validation for SOURCES_INDEX"
enforcement_level: "hard"

KL-2.4b — SOURCES_INDEX Validation Checklist

Passes if:

Schema
- [ ] JSON parses
- [ ] Matches KL-2.4 schema

Uniqueness
- [ ] source_id is unique

Optional strict referential checks (recommended)
- [ ] Any KID marked as derived/excerpting external content references a valid source_id
- [ ] If attribution_required==true, KID includes attribution in Links section

Say **“next”** for **KL-2.5 DEPRECATIONS + CHANGELOG**.

KL-2.5 DEPRECATIONS + CHANGELOG

KL-2.5 — DEPRECATIONS + CHANGELOG (required)

Piece A — Deprecations registry schema (JSON Schema)

File: `knowledge/contracts/KL-2.5_deprecations.schema.json`

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "schema_version": "1.0.0",
 "contract_id": "KL-2.5",
 "title": "DEPRECATIONS Registry Schema",
 "type": "object",
 "required": ["schema_version", "deprecated"],
 "properties": {
   "schema_version": { "type": "string", "minLength": 1 },
   "deprecated": {
     "type": "array",
     "items": { "$ref": "#/$defs/deprecation" }
   }
 },
 "$defs": {
   "deprecation": {
     "type": "object",
     "required": [
       "kind",
       "id",
       "replacement_id",
       "reason",
       "effective_date",
       "created_at",
       "owner"
     ],
     "properties": {
       "kind": {
         "type": "string",
         "enum": ["kid", "bundle", "tag", "domain", "subdomain"]
       },
       "id": { "type": "string", "minLength": 3 },
       "replacement_id": { "type": "string", "minLength": 3 },
       "reason": { "type": "string", "minLength": 3 },
       "effective_date": { "type": "string", "format": "date" },
```

```
      "allow_for_repro": {
        "type": "boolean",
        "default": true,
        "description": "If true, deprecated items may be selected only in reproducibility mode."
      },

      "notes": { "type": "string" },

      "created_at": { "type": "string", "format": "date" },
      "owner": { "type": "string", "minLength": 2 }
    },
    "additionalProperties": false
  }
 },
 "additionalProperties": false
}
```

Piece B — Deprecation rules (how selection treats deprecated things)

File: `knowledge/contracts/KL-2.5_deprecations_rules.md`

```
---
contract_id: KL-2.5a
schema_version: 1.0.0
applies_to: "knowledge/INDEX/DEPRECATIONS.json"
enforcement_level: "hard"
---

# KL-2.5a — Deprecations Rules

## Purpose
DEPRECATIONS provides the authoritative mapping from deprecated items to replacements.

## Selection behavior
- Deprecated KIDs MUST NOT be selected in normal mode.
- Deprecated KIDs MAY be selected only when:
 - run_mode == reproducibility (repro)
 - allow_for_repro == true on the deprecation record
- When a deprecated item is used in repro:
 - the selection artifact MUST include a deprecation_notice for it.

## Consistency behavior
- If a KID frontmatter includes deprecated_by:
```

- DEPRECATIONS MUST include a record kind=kid with id=<kid> and replacement_id=<deprecated_by>.
- If a bundle has deprecated_by:
- DEPRECATIONS MUST include kind=bundle mapping.
- If a tag/domain/subdomain is deprecated:
- TAXONOMY must keep alias compatibility during a migration window.

Piece C — Minimal DEPRECATIONS registry file

File: `knowledge/INDEX/DEPRECATIONS.json`

```
{
 "schema_version": "1.0.0",
 "deprecated": []
}
```

Piece D — Example DEPRECATIONS entries

File: `knowledge/INDEX/DEPRECATIONS.seed.example.json`

```
{
 "schema_version": "1.0.0",
 "deprecated": [
   {
     "kind": "kid",
     "id": "KID-IT_END_TO_END-SECURITY_FUNDAMENTALS-CONCEPT-0001",
     "replacement_id": "KID-IT_END_TO_END-SECURITY_FUNDAMENTALS-CONCEPT-0002",
     "reason": "Updated threat model framing and clarified applicability rules.",
     "effective_date": "2026-03-05",
     "allow_for_repro": true,
     "notes": "Use -0002 for all new selections; -0001 only for repro of historical runs.",
     "created_at": "2026-03-05",
     "owner": "axion"
   }
 ]
}
```

Piece E — CHANGELOG format contract

File: `knowledge/contracts/KL-2.5_changelog_contract.md`

---
contract_id: KL-2.5b
schema_version: 1.0.0
applies_to: "knowledge/INDEX/CHANGELOG.md"
enforcement_level: "soft"

KL-2.5b — Knowledge Library CHANGELOG Contract

Purpose
Track library-level changes for reproducibility and governance.

Required sections
- Unreleased
- Date-stamped entries in reverse chronological order

What must be logged
- Added/removed/deprecated KIDs
- Taxonomy changes (new tags/domains/aliases)
- Bundle changes (new bundles or deprecations)
- Source ingestion changes (new sources and reuse constraints)

Recommended entry format
- Added: KID-...
- Updated: KID-...
- Deprecated: KID-... -> KID-...
- Taxonomy: added tag <x>, alias <y -> x>
- Bundles: added/updated bundle <id>

Piece F — Minimal CHANGELOG file (scaffold)

File: `knowledge/INDEX/CHANGELOG.md`

Knowledge Library Changelog

Unreleased
- (add entries)

2026-03-05
- Initialized registries: KNOWLEDGE_INDEX, TAXONOMY, BUNDLES_INDEX, SOURCES_INDEX, DEPRECATIONS.

Piece G — Validation checklist (gate view)

File: `knowledge/contracts/KL-2.5_validation_checklist.md`

```
---
contract_id: KL-2.5c
schema_version: 1.0.0
applies_to: "KL integrity validation for DEPRECATIONS + CHANGELOG coherence"
enforcement_level: "hard"
---

# KL-2.5c — Validation Checklist

## Deprecations
- [ ] DEPRECATIONS.json parses and matches schema
- [ ] No duplicate (kind,id) pairs
- [ ] replacement_id exists (strict mode):
 - for kind=kid -> exists in KNOWLEDGE_INDEX
 - for kind=bundle -> exists in BUNDLES_INDEX
 - for kind=tag/domain/subdomain -> exists in TAXONOMY (or in alias migration set)

## Coherence with metadata
- [ ] Any KID with deprecated_by has a matching DEPRECATIONS record
- [ ] Any bundle with deprecated_by has a matching DEPRECATIONS record

## CHANGELOG presence
- [ ] CHANGELOG.md exists
```

KL-3) Knowledge Selection

KL-3) Knowledge Selection (How agents “use it” safely)

Agents should not roam. The system selects.

KL-3.1 Inputs to selection

- canonical spec signals (domain/feature requirements)
- standards snapshot
- run profile (API/Web/etc.)
- risk class
- executor type (internal vs external)
- target stack (if known)

KL-3.2 Output: KNOWLEDGE_SELECTION artifact (required)

A deterministic artifact like:

- selected KIDs (ordered)
- why each KID was selected (tags/applicability match)
- restrictions attached (use_policy, excerpt limits)
- bundle IDs used (if any)

KL-3.3 Selection rules (minimum)

- filter by executor access
- filter by applicability
- filter by maturity threshold (e.g., production requires reviewed+)

- rank by relevance (tags + domain match)
- cap selection set (avoid overload)

KL-3.1 Inputs to selection

KL-3.1 — Inputs to selection

Piece A — Selection input schema (machine-readable)

File: `knowledge/contracts/KL-3.1_selection_inputs.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "contract_id": "KL-3.1",
  "title": "Knowledge Selection Inputs Schema",
  "type": "object",
  "required": [
    "run_id",
    "run_profile",
    "risk_class",
    "executor_type",
    "min_maturity",
    "signals"
  ],
  "properties": {
    "run_id": { "type": "string", "minLength": 3 },

    "run_profile": {
      "type": "string",
      "description": "High-level build context",
      "enum": ["API", "WEB", "MOBILE", "INFRA", "DATA", "CLI", "MIXED"]
    },

    "risk_class": {
      "type": "string",
      "description": "Controls maturity thresholds and stricter gating",
      "enum": ["PROTOTYPE", "PROD", "COMPLIANCE"]
    },

    "executor_type": {
      "type": "string",
      "description": "Where the agent runs / who can see what",
      "enum": ["internal", "external"]
    },

    "min_maturity": {
```

```
  "type": "string",
  "description": "Minimum maturity allowed for selection",
  "enum": ["draft", "reviewed", "verified", "golden"]
},

"target_stack": {
  "type": "object",
  "description": "Optional stack signals if known",
  "properties": {
    "languages": { "type": "array", "items": { "type": "string" } },
    "frameworks": { "type": "array", "items": { "type": "string" } },
    "datastores": { "type": "array", "items": { "type": "string" } },
    "infra": { "type": "array", "items": { "type": "string" } }
  },
  "additionalProperties": false
},

"standards_snapshot_id": {
  "type": "string",
  "description": "Reference to resolved standards snapshot used in this run"
},

"canonical_spec_signals": {
  "type": "object",
  "description": "Signals extracted from canonical spec (domains/features/requirements)",
  "properties": {
    "domains": { "type": "array", "items": { "type": "string" } },
    "subdomains": { "type": "array", "items": { "type": "string" } },
    "features": { "type": "array", "items": { "type": "string" } },
    "tags": { "type": "array", "items": { "type": "string" } }
  },
  "additionalProperties": false
},

"bundle_requests": {
  "type": "array",
  "description": "Optional explicit bundle IDs to apply",
  "items": { "type": "string" }
},

"signals": {
  "type": "object",
  "description": "Computed selection signals used for ranking and filtering",
  "required": ["domains", "tags"],
```

```
      "properties": {
        "domains": { "type": "array", "items": { "type": "string" }, "minItems": 1 },
        "subdomains": { "type": "array", "items": { "type": "string" } },
        "tags": { "type": "array", "items": { "type": "string" }, "minItems": 1 },
        "industries": { "type": "array", "items": { "type": "string" } },
        "stack_families": { "type": "array", "items": { "type": "string" } }
      },
      "additionalProperties": false
    },

    "selection_cap": {
      "type": "integer",
      "minimum": 1,
      "default": 30,
      "description": "Hard cap to avoid overload"
    },

    "mode": {
      "type": "string",
      "enum": ["normal", "reproducibility"],
      "default": "normal"
    }
  },
  "additionalProperties": false
}
```

Piece B — Input derivation rules (where each input comes from)

File: `knowledge/contracts/KL-3.1_selection_input_derivation.md`

```
---
contract_id: KL-3.1a
schema_version: 1.0.0
applies_to: "Generation of selection inputs before knowledge selection"
enforcement_level: "hard"
---

# KL-3.1a — Selection Input Derivation Rules

## Source mapping
Selection inputs MUST be derived from these upstream sources:
```

1) canonical spec signals
- domains/subdomains/features/tags extracted from canonical spec artifacts
- used to drive domain + tag matching

2) standards snapshot
- standards_snapshot_id references the resolved standards snapshot for the run
- used to:
  - add required domains/tags (if standards imply them)
  - tighten maturity thresholds for high-risk categories

3) run profile
- API / WEB / MOBILE / etc.
- used to apply bundles and/or profile-specific weighting

4) risk class
- PROTOTYPE / PROD / COMPLIANCE
- used to set min_maturity:
  - PROTOTYPE: draft+
  - PROD: reviewed+
  - COMPLIANCE: verified+

(These are defaults; overridable by policy.)

5) executor type
- internal vs external
- used to filter by executor_access during selection

6) target stack (optional)
- used to prefer language/framework KIDs and stack-family bundles

Normalization before selection
Before selection runs:
- domains/subdomains MUST be canonical slugs (TAXONOMY)
- tags MUST be canonical tags (alias-resolved via TAXONOMY.tag_aliases)
- duplicate tags/domains removed; sort lexicographically for determinism

---

Piece C — Minimal input template (scaffold)

File: `knowledge/templates/KNOWLEDGE_SELECTION_INPUT.json`

```
{
  "schema_version": "1.0.0",
  "run_id": "RUN-000000",
```

```
  "run_profile": "API",
  "risk_class": "PROTOTYPE",
  "executor_type": "internal",
  "min_maturity": "draft",
  "signals": {
    "domains": ["security_fundamentals"],
    "subdomains": [],
    "tags": ["auth", "validation"],
    "industries": [],
    "stack_families": []
  },
  "selection_cap": 30,
  "mode": "normal"
}
```

Piece D — Validation checklist (gate view)

File: `knowledge/contracts/KL-3.1_validation_checklist.md`

```
---
contract_id: KL-3.1b
schema_version: 1.0.0
applies_to: "KL selection input validation"
enforcement_level: "hard"
---

# KL-3.1b — Selection Inputs Validation Checklist

Passes if:

## Schema
- [ ] JSON parses
- [ ] Matches KL-3.1 schema

## Canonicalization (strict mode)
- [ ] signals.domains exist in TAXONOMY.domains
- [ ] signals.subdomains exist under the referenced domains (if provided)
- [ ] signals.tags are canonical or alias-resolved to canonical tags

## Determinism
- [ ] domains and tags are de-duplicated and sorted
- [ ] selection_cap is set (defaults allowed)
```

KL-3.2 Output: KNOWLEDGE_SELECTION

KL-3.2 — Output: KNOWLEDGE_SELECTION artifact (required)

Piece A — Artifact schema (JSON Schema)

File: `knowledge/contracts/KL-3.2_knowledge_selection.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "artifact_id": "KNOWLEDGE_SELECTION",
  "title": "KNOWLEDGE_SELECTION Artifact Schema",
  "type": "object",
  "required": [
    "schema_version",
    "run_id",
    "generated_at",
    "mode",
    "inputs",
    "bundles_used",
    "selected",
    "cap",
    "notes"
  ],
  "properties": {
    "schema_version": { "type": "string", "minLength": 1 },
    "run_id": { "type": "string", "minLength": 3 },
    "generated_at": { "type": "string", "format": "date-time" },

    "mode": {
      "type": "string",
      "enum": ["normal", "reproducibility"]
    },

    "inputs": {
      "type": "object",
      "description": "A frozen copy of KL-3.1 inputs used for selection",
      "required": ["run_profile", "risk_class", "executor_type", "min_maturity", "signals"],
      "properties": {
        "run_profile": { "type": "string" },
        "risk_class": { "type": "string" },
        "executor_type": { "type": "string" },
```

```
      "min_maturity": { "type": "string" },
      "target_stack": { "type": "object" },
      "standards_snapshot_id": { "type": "string" },
      "signals": { "type": "object" }
    },
    "additionalProperties": true
  },

  "bundles_used": {
    "type": "array",
    "description": "Bundle IDs applied (in application order)",
    "items": { "type": "string" }
  },

  "selected": {
    "type": "array",
    "description": "Ordered list of selected KIDs with reasons + restrictions",
    "minItems": 0,
    "items": { "$ref": "#/$defs/selected_item" }
  },

  "cap": { "type": "integer", "minimum": 1 },

  "notes": {
    "type": "array",
    "items": { "type": "string" }
  }
},

"$defs": {
  "selected_item": {
    "type": "object",
    "required": [
      "kid",
      "title",
      "type",
      "path",
      "why_selected",
      "restrictions"
    ],
    "properties": {
      "kid": { "type": "string", "minLength": 8 },
      "title": { "type": "string", "minLength": 3 },
      "type": { "type": "string" },
```

```
"path": { "type": "string", "minLength": 5 },

"why_selected": {
  "type": "object",
  "required": ["matches", "score"],
  "properties": {
    "matches": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["kind", "value"],
        "properties": {
          "kind": {
            "type": "string",
            "enum": ["domain", "subdomain", "tag", "bundle", "profile", "stack_family"]
          },
          "value": { "type": "string" }
        },
        "additionalProperties": false
      }
    },
    "applicability": {
      "type": "string",
      "description": "Optional short note about 'When to use' match"
    },
    "score": { "type": "number", "minimum": 0 }
  },
  "additionalProperties": false
},

"restrictions": {
  "type": "object",
  "required": ["use_policy", "executor_access", "allowed_excerpt", "maturity"],
  "properties": {
    "use_policy": {
      "type": "string",
      "enum": ["pattern_only", "reusable_with_allowlist", "restricted_internal_only"]
    },
    "executor_access": {
      "type": "string",
      "enum": ["internal_only", "internal_and_external"]
    },
    "allowed_excerpt": {
      "type": "object",
```

```
            "required": ["max_words", "max_lines"],
            "properties": {
              "max_words": { "type": "integer", "minimum": 0 },
              "max_lines": { "type": "integer", "minimum": 0 }
            },
            "additionalProperties": false
          },
          "maturity": {
            "type": "string",
            "enum": ["draft", "reviewed", "verified", "golden"]
          }
        },
        "additionalProperties": false
      },

      "deprecation_notice": {
        "type": "object",
        "description": "Present only if selected item is deprecated (repro mode)",
        "required": ["replacement_id", "reason"],
        "properties": {
          "replacement_id": { "type": "string" },
          "reason": { "type": "string" }
        },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece B — Output rules (ordering, determinism, restrictions)

File: `knowledge/contracts/KL-3.2_knowledge_selection_rules.md`

```
---
contract_id: KL-3.2a
schema_version: 1.0.0
applies_to: "KNOWLEDGE_SELECTION artifact generation"
enforcement_level: "hard"
```

KL-3.2a — KNOWLEDGE_SELECTION Output Rules

Deterministic ordering
The "selected" list MUST be ordered deterministically:
1) bundle contribution order (if bundles applied)
2) then ranked relevance (KL-3.3 ranking rules)
3) tie-breakers:
  - pillar
  - primary domain
  - type
  - kid

What must be included per selected item
Each selected item MUST include:
- kid, title, type, path (from KNOWLEDGE_INDEX)
- why_selected.matches (domain/tag/bundle/profile/stack_family hits)
- why_selected.score (numeric)
- restrictions snapshot:
  - use_policy, executor_access, allowed_excerpt, maturity

Cap enforcement
- "cap" MUST equal the applied selection_cap.
- selected.length MUST be <= cap.
- If candidate set > cap:
  - truncate after sorting
  - include note "cap_applied" with counts.

Repro mode
If mode == reproducibility:
- deprecated items may appear only if allow_for_repro=true.
- any deprecated selected item MUST include deprecation_notice.

Piece C — Minimal artifact template (scaffold)

File: `knowledge/templates/KNOWLEDGE_SELECTION.artifact.json`

```
{
  "schema_version": "1.0.0",
  "run_id": "RUN-000000",
  "generated_at": "2026-03-05T00:00:00Z",
```

```
  "mode": "normal",
  "inputs": {
    "run_profile": "API",
    "risk_class": "PROTOTYPE",
    "executor_type": "internal",
    "min_maturity": "draft",
    "signals": { "domains": ["security_fundamentals"], "tags": ["auth", "validation"] }
  },
  "bundles_used": [],
  "selected": [],
  "cap": 30,
  "notes": []
}
```

Piece D — Validation checklist (gate view)

File: `knowledge/contracts/KL-3.2_validation_checklist.md`

```
---
contract_id: KL-3.2b
schema_version: 1.0.0
applies_to: "KL selection output validation"
enforcement_level: "hard"
---

# KL-3.2b — KNOWLEDGE_SELECTION Validation Checklist

Passes if:

## Schema
- [ ] JSON parses
- [ ] Matches KL-3.2 schema

## Referential integrity (strict mode)
- [ ] Every selected.kid exists in KNOWLEDGE_INDEX
- [ ] selected.title/type/path match the registry for that kid

## Restrictions snapshot
- [ ] Each selected item contains restrictions: use_policy, executor_access, allowed_excerpt, maturity

## Cap
```

- [ ] selected.length <= cap

Deprecations
- [ ] If any selected item is deprecated:
  - [ ] mode == reproducibility
  - [ ] deprecation_notice present

KL-3.3 Selection rules

KL-3.3 — Selection rules (minimum)

Piece A — Selection algorithm contract (filter → rank → cap)

File: `knowledge/contracts/KL-3.3_selection_rules.md`

```
---
contract_id: KL-3.3
schema_version: 1.0.0
applies_to: "Knowledge selection engine"
enforcement_level: "hard"
---

# KL-3.3 — Selection Rules (Minimum)

Selection MUST run in three phases, in this exact order:
1) Filter (hard exclusions)
2) Rank (relevance scoring)
3) Cap (limit and finalize ordering)

---

## 1) FILTER (hard exclusions)

### F1 — executor access
- If executor_type == external:
  - exclude any KID where executor_access == internal_only

### F2 — maturity threshold
- Exclude any KID where maturity < min_maturity
  - ordering: draft < reviewed < verified < golden

### F3 — deprecations
- If mode == normal:
  - exclude any deprecated KID (DEPRECATIONS.kind=kid where id == kid)
- If mode == reproducibility:
  - allow deprecated KID only if allow_for_repro == true

### F4 — applicability (minimum)
A KID is applicable if it matches at least ONE of:
- domain match: any selection input domain appears in KID domains
- tag match: any selection input tag appears in KID tags
```

- bundle inclusion: KID is explicitly included via bundles_used
If none match → exclude.

---

2) RANK (relevance scoring)

Compute a numeric score per remaining KID as:

```
score =
  (domain_hit * 5) +
  (subdomain_hit * 3) +
  (tag_hit_count * 2) +
  (profile_hit * 1) +
  (stack_family_hit * 1) +
  (maturity_bonus)
```

Where:
- domain_hit is 1 if any domain matches else 0
- subdomain_hit is 1 if any subdomain matches else 0
- tag_hit_count is the number of matched tags (cap at 5)
- profile_hit is 1 if the KID tags/domains indicate run_profile relevance (optional mapping)
- stack_family_hit is 1 if KID tags match target_stack families (optional mapping)
- maturity_bonus:
  - draft: 0
  - reviewed: 0.5
  - verified: 1.0
  - golden: 1.5

---

3) CAP (limit and finalize)

C1 — selection cap
- selected.length MUST be <= selection_cap

C2 — final ordering
Primary ordering:
- higher score first

Tie-breakers (deterministic):
1) pillar (lexicographic)
2) primary domain (domains[0])
3) type (lexicographic)

4) kid (lexicographic)

C3 — bundle precedence
If bundles_used is non-empty:
- Apply bundles in order.
- Any KIDs contributed by bundles must appear first (preserve bundle order),
  then fill remaining slots by ranked candidates not already included.

Piece B — “Applicability extraction” rule (how to infer applicability safely)

File: `knowledge/contracts/KL-3.3_applicability_extraction.md`

```
---
contract_id: KL-3.3a
schema_version: 1.0.0
applies_to: "Determining applicability match for why_selected.applicability"
enforcement_level: "soft"
---

# KL-3.3a — Applicability Extraction

## Purpose
Provide a short human-readable reason that ties selection to the KID’s "When to use" section,
without letting agents roam or reinterpret the library.

## Rule
The selector MAY extract a brief applicability note by:
- referencing matched domain/tags
- referencing "When to use" bullets ONLY as short paraphrase (no verbatim)

## Format
why_selected.applicability should be:
- 1 sentence max
- no quotes
- no raw copying
```

Piece C — Validation checklist (gate view)

File: `knowledge/contracts/KL-3.3_validation_checklist.md`

---
contract_id: KL-3.3b
schema_version: 1.0.0
applies_to: "KL selection rules validation (engine behavior)"
enforcement_level: "hard"

KL-3.3b — Selection Rules Validation Checklist

Passes if:

Filter compliance
- [ ] External executor never receives internal_only items
- [ ] maturity threshold enforced
- [ ] deprecated items excluded in normal mode
- [ ] deprecated items only allowed in repro mode when allow_for_repro=true
- [ ] applicability filter applied (domain/tag/bundle)

Rank compliance
- [ ] scoring function applied consistently
- [ ] tie-breakers deterministic

Cap compliance
- [ ] selected.length <= selection_cap
- [ ] bundle ordering preserved when bundles are used

KL-4) Usage Rules

KL-4) Usage Rules (what agents may do)

This is where your plagiarism/IP policy hooks in.

KL-4.1 Default rule

- Knowledge items are **pattern-only** unless explicitly allowlisted.

KL-4.2 Allowed behaviors

- derive requirements and constraints
- follow procedures and checklists
- re-express concepts in original language
- use examples to understand format (not copy)

KL-4.3 Restricted behaviors

- copying code blocks or long prose unless `reusable_with_allowlist`
- using internal-only items with external agents
- including restricted content inside kits for external execution

KL-4.4 Mandatory citation behavior

When a KID is used, agent must emit:

- `knowledge_citations: [KID-...]` in render envelope / run log
- if reuse: update reuse_log

KL-4.1 Default rule

KL-4.1 — Default rule (pattern-only unless allowlisted)

Piece A — Default usage policy contract

File: `knowledge/contracts/KL-4.1_default_usage_rule.md`

```
---
contract_id: KL-4.1
schema_version: 1.0.0
applies_to: "All agent consumption of Knowledge Library items"
enforcement_level: "hard"
---

# KL-4.1 — Default Usage Rule

## Rule
Knowledge Items are **pattern-only by default**.

This means:
- Agents may use KIDs to derive constraints, requirements, and approaches.
- Agents must **re-express** content in original language.
- Agents must NOT copy long prose or code from KIDs into outputs.

## Allowlisting
The only exception is when:
- KID.use_policy == reusable_with_allowlist, AND
- allowed_excerpt.max_words OR allowed_excerpt.max_lines > 0, AND
- the output includes a reuse_log entry (KL-4.4 / KL-5.3).

## Determinism
When a KID is used, the system MUST record its KID ID in knowledge_citations
even if no excerpt is copied verbatim.

## Non-negotiables
- No verbatim copying beyond allowed_excerpt.
- No external executor access to internal_only items.
- No exporting restricted content inside external kits.
```

Piece B — Enforcement checklist (gate view)

File: `knowledge/contracts/KL-4.1_validation_checklist.md`

---
contract_id: KL-4.1a
schema_version: 1.0.0
applies_to: "KL usage enforcement validation"
enforcement_level: "hard"

KL-4.1a — Default Usage Rule Validation Checklist

Passes if:

- [ ] For any KID used, the output/run log includes knowledge_citations containing that KID
- [ ] If any verbatim excerpt appears:
  - [ ] KID.use_policy == reusable_with_allowlist
  - [ ] excerpt length within allowed_excerpt limits
  - [ ] reuse_log entry exists
- [ ] No internal_only KID is present in any external executor context

KL-4.2 Allowed behaviors

KL-4.2 — Allowed behaviors

Piece A — Allowed behavior contract (what agents MAY do)

File: `knowledge/contracts/KL-4.2_allowed_behaviors.md`

```
---
contract_id: KL-4.2
schema_version: 1.0.0
applies_to: "Agent behavior when consuming KIDs"
enforcement_level: "hard"
---

# KL-4.2 — Allowed Behaviors

Agents MAY do the following when a KID is selected:

## 1) Derive requirements and constraints
- Convert KID guidance into:
  - acceptance criteria
  - non-functional requirements
  - interface constraints
  - validation rules
- Output must be expressed in the agent’s own words (no long copying).

## 2) Follow procedures and checklists
- Use procedure steps to guide execution.
- Use checklists as gate criteria (pass/fail items).
- Agents may reformat lists for their target artifact format, but must preserve meaning.

## 3) Re-express concepts in original language
- Concepts may be paraphrased into:
  - summaries
  - explanations
  - rationale sections
- The output should be shorter than the source and not mirror phrasing.

## 4) Use examples for structure learning (not copying)
- Examples may be used to learn:
  - formatting
  - section ordering
  - naming conventions
```

  - minimal scaffolds
- Example content must not be copied verbatim unless allowlisted.

5) Use references for defaults and enumerations (with policy)
- Reference KIDs may supply:
  - default values
  - parameter lists
  - safe ranges
- If values are transferred into outputs, the KID must be cited.

6) Emit citations
- Every use of a KID MUST emit knowledge_citations (KL-4.4).

Piece B — “Paraphrase safe-mode” operational rule (how to avoid copy-donor behavior)

File: `knowledge/contracts/KL-4.2_paraphrase_safe_mode.md`

---
contract_id: KL-4.2a
schema_version: 1.0.0
applies_to: "Any agent output derived from KIDs"
enforcement_level: "soft"

KL-4.2a — Paraphrase Safe-Mode

When drawing from KIDs (pattern_only default), agents SHOULD:
- compress: produce a shorter version than the KID section
- transform: change structure (bullets → sentences or vice versa)
- reframe: use different ordering of ideas
- localize: tie to the current spec/run context

Agents SHOULD NOT:
- reuse distinctive phrasing
- keep identical bullet ordering across 5+ bullets
- copy code blocks or paragraph chunks

Piece C — Validation checklist (gate view)

File: `knowledge/contracts/KL-4.2_validation_checklist.md`

---
contract_id: KL-4.2b
schema_version: 1.0.0
applies_to: "KL behavior validation"
enforcement_level: "hard"

KL-4.2b — Allowed Behaviors Validation Checklist

Passes if:

- [ ] Outputs derived from pattern_only KIDs contain no long verbatim excerpts
- [ ] Procedures/checklists are applied as guidance, not pasted wholesale as donor text
- [ ] Examples are used for structure only unless allowlisted
- [ ] knowledge_citations is present when any KID influences output

KL-4.3 Restricted behaviors

KL-4.3 — Restricted behaviors

Piece A — Restricted behavior contract (what agents MUST NOT do)

File: `knowledge/contracts/KL-4.3_restricted_behaviors.md`

```
---
contract_id: KL-4.3
schema_version: 1.0.0
applies_to: "Agent behavior when consuming KIDs"
enforcement_level: "hard"
---

# KL-4.3 — Restricted Behaviors

Agents MUST NOT do the following:

## 1) Copying beyond policy limits
- MUST NOT copy code blocks or long prose unless:
  - KID.use_policy == reusable_with_allowlist, AND
  - excerpt is within allowed_excerpt limits, AND
  - reuse_log entry is emitted (KL-4.4 / KL-5.3)

## 2) Leaking internal-only items to external executors
If executor_type == external:
- MUST NOT use any KID where executor_access == internal_only
- MUST NOT include internal-only content in artifacts or kits exported for external execution

## 3) Including restricted content inside external kits
- MUST NOT package KID content (full text or excerpts) into external kits unless explicitly
permitted.
- Even when permitted, include only excerpts within allowed limits and record reuse.

## 4) Using examples as donors
- Examples are "structure learning only" unless allowlisted.
- MUST NOT paste example content into outputs as-is.

## 5) Bypassing selection
- Agents MUST NOT “roam” the library.
- Only KIDs present in KNOWLEDGE_SELECTION may influence outputs.
- Direct reads outside the selection list is a hard violation.
```

6) Smuggling via paraphrase
- MUST NOT produce near-verbatim paraphrases that preserve distinctive phrasing and ordering.
- If output similarity is high, treat as copying and block (KL-5.3).

Piece B — Restricted behaviors taxonomy (reason codes)

File: `knowledge/contracts/KL-4.3_restriction_reason_codes.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "KL-4.3a",
  "restriction_reason_codes": [
    { "code": "KL-R01", "name": "Verbatim beyond excerpt limits" },
    { "code": "KL-R02", "name": "Internal-only item used by external executor" },
    { "code": "KL-R03", "name": "Restricted content exported in external kit" },
    { "code": "KL-R04", "name": "Example used as copy donor" },
    { "code": "KL-R05", "name": "Bypassed selection (roamed library)" },
    { "code": "KL-R06", "name": "High-similarity paraphrase (near-copy)" }
  ]
}
```

Piece C — Validation checklist (gate view)

File: `knowledge/contracts/KL-4.3_validation_checklist.md`

```
---
contract_id: KL-4.3b
schema_version: 1.0.0
applies_to: "KL restricted behaviors validation"
enforcement_level: "hard"
---

# KL-4.3b — Restricted Behaviors Validation Checklist

Passes if:

- [ ] No output contains verbatim excerpts beyond allowed_excerpt limits
- [ ] No internal_only KID is used when executor_type == external
- [ ] No restricted KID content is present in external kit exports
```

- [ ] No KID outside KNOWLEDGE_SELECTION is cited or used
- [ ] No example KID content is copied unless allowlisted
- [ ] Similarity checks (if enabled) do not trigger KL-R06

KL-4.4 Mandatory citation behavior

KL-4.4 — Mandatory citation behavior

Piece A — Citation + reuse emission contract

File: `knowledge/contracts/KL-4.4_citation_behavior.md`

```
---
contract_id: KL-4.4
schema_version: 1.0.0
applies_to: "Any artifact/render output influenced by KIDs"
enforcement_level: "hard"
---

# KL-4.4 — Mandatory Citation Behavior

## Rule
If a KID influences an output in any way, the agent MUST emit:

1) knowledge_citations: [KID-...]
2) reuse_log entries ONLY if verbatim reuse occurs (allowlisted only)

## knowledge_citations (always required when used)
- Must list all KIDs that influenced:
  - requirements
  - constraints
  - checklists/procedures followed
  - defaults copied from references
- Must include KIDs even when content is paraphrased.

## reuse_log (required only for allowlisted verbatim reuse)
A reuse_log entry MUST exist if:
- any excerpt (prose/code/table row) is copied verbatim, AND
- allowed by use_policy + allowed_excerpt

If reuse occurs:
- The exact excerpt length MUST be recorded (words/lines)
- The target location MUST be recorded (artifact + section)
- The system MUST verify limits before allowing export/close.

## Prohibitions
- reuse_log MUST NOT be emitted for pattern_only usage unless excerpting occurred.
- If excerpting occurred while use_policy != reusable_with_allowlist, block the run.
```

Piece B — Render envelope fields (where citations live)

File: `knowledge/contracts/KL-4.4_render_envelope_addendum.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "KL-4.4a",
  "render_envelope_addendum": {
    "knowledge_citations": {
      "type": "array",
      "items": { "type": "string" },
      "required_when": "any KID used"
    },
    "reuse_log": {
      "type": "array",
      "items": { "$ref": "KL-4.4_reuse_log_entry.schema.json" },
      "required_when": "any verbatim reuse occurs"
    }
  }
}
```

Piece C — Reuse log entry schema (JSON Schema)

File: `knowledge/contracts/KL-4.4_reuse_log_entry.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "contract_id": "KL-4.4b",
  "title": "Reuse Log Entry Schema",
  "type": "object",
  "required": [
    "kid",
    "reuse_type",
    "excerpt_size",
    "target",
    "justification",
    "recorded_at"
  ],
```

```
  "properties": {
    "kid": { "type": "string", "minLength": 8 },

    "reuse_type": {
      "type": "string",
      "enum": ["prose_excerpt", "code_excerpt", "table_excerpt"]
    },

    "excerpt_size": {
      "type": "object",
      "required": ["words", "lines"],
      "properties": {
        "words": { "type": "integer", "minimum": 0 },
        "lines": { "type": "integer", "minimum": 0 }
      },
      "additionalProperties": false
    },

    "target": {
      "type": "object",
      "required": ["artifact_id", "path", "section"],
      "properties": {
        "artifact_id": { "type": "string", "minLength": 2 },
        "path": { "type": "string", "minLength": 2 },
        "section": { "type": "string", "minLength": 1 }
      },
      "additionalProperties": false
    },

    "justification": {
      "type": "string",
      "minLength": 3,
      "description": "Why reuse was necessary vs paraphrase"
    },

    "recorded_at": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": false
}
```

Piece D — Minimal examples (what agents emit)

File: `knowledge/examples/KL-4.4_citations_example.json`

```
{
  "knowledge_citations": [
    "KID-IT_END_TO_END-SECURITY_FUNDAMENTALS-CHECKLIST-0001",
    "KID-IT_END_TO_END-SECURITY_FUNDAMENTALS-CONCEPT-0002"
  ],
  "reuse_log": []
}
```

File: `knowledge/examples/KL-4.4_reuse_log_example.json`

```
{
  "knowledge_citations": [
    "KID-LANGUAGES_AND_LIBRARIES-TYPESCRIPT-REFERENCE-0003"
  ],
  "reuse_log": [
    {
      "kid": "KID-LANGUAGES_AND_LIBRARIES-TYPESCRIPT-REFERENCE-0003",
      "reuse_type": "table_excerpt",
      "excerpt_size": { "words": 22, "lines": 3 },
      "target": {
        "artifact_id": "API_SPEC",
        "path": "runs/RUN-000123/artifacts/API_SPEC.md",
        "section": "Defaults"
      },
      "justification": "Needed exact parameter default values to avoid ambiguity.",
      "recorded_at": "2026-03-05T00:00:00Z"
    }
  ]
}
```

Piece E — Validation checklist (gate view)

File: `knowledge/contracts/KL-4.4_validation_checklist.md`

```
---
contract_id: KL-4.4c
schema_version: 1.0.0
applies_to: "KL citation + reuse validation"
enforcement_level: "hard"
```

KL-4.4c — Validation Checklist

Passes if:

knowledge_citations
- [ ] knowledge_citations exists whenever any KID is used
- [ ] every cited KID exists in KNOWLEDGE_INDEX (strict mode)
- [ ] no KID outside KNOWLEDGE_SELECTION is cited (hard)

reuse_log
- [ ] reuse_log exists if verbatim excerpts exist in output
- [ ] every reuse_log entry references a KID with use_policy == reusable_with_allowlist
- [ ] excerpt_size within allowed_excerpt limits for that KID
- [ ] target fields are present (artifact_id/path/section)

KL-5) Enforcement Gates

KL-5) Enforcement Gates (Knowledge safety and integrity)

These make the library “real” and safe.

KL-5.1 Integrity gates

- **KL-GATE-01**: every referenced KID exists in KNOWLEDGE_INDEX
- **KL-GATE-02**: all KID metadata valid (type, tags, policies)
- **KL-GATE-03**: no deprecated KIDs used unless explicitly allowed

KL-5.2 Access gates

- **KL-GATE-04**: external executor cannot access restricted KIDs
- **KL-GATE-05**: kit export for external use excludes restricted content

KL-5.3 Plagiarism/reuse gates

- **KL-GATE-06**: if output contains allowlisted excerpts, reuse_log must include entry
- **KL-GATE-07**: block verbatim copying beyond excerpt limits

KL-5.4 Quality gates (optional but strong)

- **KL-GATE-08**: production runs require maturity >= reviewed for selected KIDs

KL-5.1 Integrity gates

KL-5.1 — Integrity gates

Piece A — Gate definitions (IDs + intent)

File: `knowledge/contracts/KL-5.1_integrity_gates.md`

```
---
contract_id: KL-5.1
schema_version: 1.0.0
applies_to: "Knowledge Library enforcement"
enforcement_level: "hard"
---

# KL-5.1 — Integrity Gates

## KL-GATE-01 — Referenced KIDs exist
**Rule:** Every KID referenced in:
- KNOWLEDGE_SELECTION.selected
- knowledge_citations
- reuse_log
MUST exist in KNOWLEDGE_INDEX.

**Fail reason code:** KL-GATE-01

---

## KL-GATE-02 — KID metadata valid
**Rule:** Every selected KID MUST pass KL-1.3 metadata validation.

**Fail reason code:** KL-GATE-02

---

## KL-GATE-03 — Deprecated avoidance
**Rule:** Deprecated KIDs MUST NOT be used unless:
- mode == reproducibility AND allow_for_repro==true.

If used in repro, output MUST include deprecation_notice.

**Fail reason code:** KL-GATE-03
```

Piece B — Machine-readable gate spec (for gate runner)

File: `knowledge/contracts/KL-5.1_integrity_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "KL-5.1a",
  "gates": [
    {
      "gate_id": "KL-GATE-01",
      "name": "Referenced KIDs exist",
      "severity": "hard_stop",
      "checks": [
        "all(knowledge_selection.selected[].kid in knowledge_index.items[].kid)",
        "all(render.knowledge_citations[] in knowledge_index.items[].kid)",
        "all(render.reuse_log[].kid in knowledge_index.items[].kid)"
      ]
    },
    {
      "gate_id": "KL-GATE-02",
      "name": "KID metadata valid",
      "severity": "hard_stop",
      "checks": [
        "all(selected_kids[].frontmatter matches KL-1.3 schema)"
      ]
    },
    {
      "gate_id": "KL-GATE-03",
      "name": "Deprecated avoidance",
      "severity": "hard_stop",
      "checks": [
        "if mode==normal then none(selected_kids[].kid in deprecations.kind:kid)",
        "if mode==reproducibility then all(deprecated_selected have deprecation_notice)"
      ]
    }
  ]
}
```

Piece C — Evidence format (what a failure report must include)

File: `knowledge/contracts/KL-5.1_gate_evidence_format.md`

---
contract_id: KL-5.1b
schema_version: 1.0.0
applies_to: "Gate failure reporting for KL-5.1"
enforcement_level: "hard"

KL-5.1b — Gate Evidence Format

Any KL-5.1 gate failure report MUST include:
- gate_id
- failing_kids (list)
- locations (where referenced: selection/citations/reuse_log)
- reason (human readable)
- remediation (what to change)

Example:
- gate_id: KL-GATE-01
- failing_kids: ["KID-...-9999"]
- locations: ["knowledge_citations", "KNOWLEDGE_SELECTION.selected[3]"]
- remediation: "Add KID to KNOWLEDGE_INDEX or remove reference."

Piece D — Validation checklist (gate view)

File: `knowledge/contracts/KL-5.1_validation_checklist.md`

---
contract_id: KL-5.1c
schema_version: 1.0.0
applies_to: "KL integrity gate validation"
enforcement_level: "hard"

KL-5.1c — Integrity Gates Validation Checklist

- [ ] KL-GATE-01 checks exist for selection + citations + reuse_log
- [ ] KL-GATE-02 references KL-1.3 metadata validation
- [ ] KL-GATE-03 references DEPRECATIONS and mode handling
- [ ] Evidence format includes gate_id + failing_kids + locations + remediation

KL-5.2 Access gates

KL-5.2 — Access gates

Piece A — Gate definitions (IDs + intent)

File: `knowledge/contracts/KL-5.2_access_gates.md`

```
---
contract_id: KL-5.2
schema_version: 1.0.0
applies_to: "Knowledge selection + kit export"
enforcement_level: "hard"
---

# KL-5.2 — Access Gates

## KL-GATE-04 — External executor cannot access restricted KIDs
**Rule:** If executor_type == external, then:
- NO selected KID may have executor_access == internal_only.

**Fail reason code:** KL-GATE-04

---

## KL-GATE-05 — External kit export excludes restricted content
**Rule:** If kit_export.target_executor == external:
- the kit MUST NOT contain:
  - full text of any internal_only KID
  - any verbatim excerpts from internal_only KIDs
  - any KID content with use_policy == restricted_internal_only
- Even for reusable_with_allowlist, only include excerpts within allowed_excerpt limits.

**Fail reason code:** KL-GATE-05
```

Piece B — Machine-readable gate spec (for gate runner)

File: `knowledge/contracts/KL-5.2_access_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "KL-5.2a",
  "gates": [
```

```
    {
      "gate_id": "KL-GATE-04",
      "name": "External executor cannot access internal_only KIDs",
      "severity": "hard_stop",
      "checks": [
        "if selection.inputs.executor_type==external then
none(selection.selected[].restrictions.executor_access==internal_only)"
      ]
    },
    {
      "gate_id": "KL-GATE-05",
      "name": "External kit export excludes restricted content",
      "severity": "hard_stop",
      "checks": [
        "if kit_export.target_executor==external then none(kit.contains_internal_only_kid_text)",
        "if kit_export.target_executor==external then
none(kit.contains_restricted_internal_only_content)",
        "if kit_export.target_executor==external then all(embedded_excerpts within
allowed_excerpt limits)"
      ]
    }
  ]
}
```

Piece C — “Kit content classification” contract (what counts as restricted)

File: `knowledge/contracts/KL-5.2_kit_content_classification.md`

```
---
contract_id: KL-5.2b
schema_version: 1.0.0
applies_to: "Determining what content is allowed inside external kits"
enforcement_level: "hard"
---

# KL-5.2b — Kit Content Classification

## Content types
When packaging a kit, classify any knowledge-derived content as:

1) **citation_only**
- KID referenced, no excerpt included.
```

2) **paraphrase**
- rewritten content that does not preserve distinctive phrasing.
- allowed for external kits if the underlying KID was accessible to the executor.

3) **excerpt**
- verbatim text/code/table content from a KID.
- allowed ONLY if:
  - use_policy == reusable_with_allowlist
  - excerpt is within allowed_excerpt
  - reuse_log entry exists

Hard bans for external kits
If target_executor == external:
- Ban any excerpt or full text from:
  - executor_access == internal_only
  - use_policy == restricted_internal_only

Proof of compliance
Kit export MUST include an export_report listing:
- KIDs cited
- KIDs excerpted + excerpt sizes
- blocked items (if any) + reason codes

Piece D — Evidence format (what a failure report must include)

File: `knowledge/contracts/KL-5.2_gate_evidence_format.md`

---
contract_id: KL-5.2c
schema_version: 1.0.0
applies_to: "Gate failure reporting for KL-5.2"
enforcement_level: "hard"

KL-5.2c — Access Gate Evidence Format

Any KL-5.2 gate failure report MUST include:
- gate_id
- executor_type / target_executor
- offending_kids
- offending_paths (where content appears in kit/artifacts)

- reason code
- remediation

Example remediation:
- "Remove internal_only KIDs from selection"
- "Strip restricted content from kit export"
- "Replace excerpt with paraphrase + citations"

Piece E — Validation checklist (gate view)

File: `knowledge/contracts/KL-5.2_validation_checklist.md`

```
---
contract_id: KL-5.2d
schema_version: 1.0.0
applies_to: "KL access gate validation"
enforcement_level: "hard"
---

# KL-5.2d — Access Gates Validation Checklist

- [ ] KL-GATE-04 blocks internal_only KIDs for external executor selection
- [ ] KL-GATE-05 blocks restricted content in external kit exports
- [ ] Classification rules define citation_only / paraphrase / excerpt
- [ ] Export requires an export_report with cited + excerpted items
```

KL-5.3 Plagiarism/reuse gates

KL-5.3 — Plagiarism / reuse gates

Piece A — Gate definitions (IDs + intent)

File: `knowledge/contracts/KL-5.3_reuse_gates.md`

```
---
contract_id: KL-5.3
schema_version: 1.0.0
applies_to: "Outputs + kit packaging when KIDs are used"
enforcement_level: "hard"
---

# KL-5.3 — Plagiarism / Reuse Gates

## KL-GATE-06 — Allowlisted reuse requires reuse_log
**Rule:** If output contains any verbatim excerpt from a KID:
- reuse_log MUST include an entry for that KID, and
- KID.use_policy MUST be reusable_with_allowlist.

**Fail reason code:** KL-GATE-06

---

## KL-GATE-07 — Block verbatim copying beyond excerpt limits
**Rule:** For every reuse_log entry:
- excerpt_size MUST be <= KID.allowed_excerpt (max_words/max_lines).
If output contains verbatim text beyond allowed limits → block.

**Fail reason code:** KL-GATE-07

---

## Optional hardening
If similarity detection is enabled:
- high-similarity outputs to KID content trigger failure (KL-R06)
even if not exact verbatim.
```

Piece B — Machine-readable gate spec (for gate runner)

File: `knowledge/contracts/KL-5.3_reuse_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "KL-5.3a",
  "gates": [
    {
      "gate_id": "KL-GATE-06",
      "name": "Allowlisted reuse requires reuse_log",
      "severity": "hard_stop",
      "checks": [
        "if output.contains_verbatim_excerpts then reuse_log.exists",
        "all(reuse_log[].kid has use_policy==reusable_with_allowlist)"
      ]
    },
    {
      "gate_id": "KL-GATE-07",
      "name": "Block copying beyond excerpt limits",
      "severity": "hard_stop",
      "checks": [
        "all(reuse_log[].excerpt_size.words <= kid.allowed_excerpt.max_words)",
        "all(reuse_log[].excerpt_size.lines <= kid.allowed_excerpt.max_lines)",
        "output.verbatim_excerpt_totals_per_kid <= allowed_excerpt (strict)"
      ]
    }
  ]
}
```

Piece C — Verbatim detection contract (what counts as “verbatim excerpt”)

File: `knowledge/contracts/KL-5.3_verbatim_detection.md`

```
---
contract_id: KL-5.3b
schema_version: 1.0.0
applies_to: "Detecting reuse in outputs"
enforcement_level: "hard"
---

# KL-5.3b — Verbatim Detection

## Definition
```

An output contains a verbatim excerpt if ANY of the following are true:
- A contiguous sequence of >= 12 words matches a KID exactly, OR
- >= 3 consecutive lines match a KID exactly, OR
- A code block matches a KID code block with only whitespace changes.

Aggregation
- Excerpts are measured per KID.
- If multiple excerpts exist for the same KID:
  - total_words = sum(excerpt words)
  - total_lines = sum(excerpt lines)
- Totals must remain within allowed_excerpt.

Allowed transformations that still count as verbatim
- whitespace changes
- formatting-only changes (markdown bullets vs numbered)
- variable renaming without structural change (for code)

Exclusions
- KID IDs themselves (KID-...) do not count as excerpts.
- Short common phrases under 12 words do not count.

Piece D — Evidence format (failure report requirements)

File: `knowledge/contracts/KL-5.3_gate_evidence_format.md`

---
contract_id: KL-5.3c
schema_version: 1.0.0
applies_to: "Gate failure reporting for KL-5.3"
enforcement_level: "hard"

KL-5.3c — Reuse Gate Evidence Format

A KL-5.3 failure report MUST include:
- gate_id
- offending_kid
- excerpt_measurement (words/lines)
- allowed_excerpt (words/lines)
- locations (artifact + section/path)
- remediation

Remediation examples:
- "Replace excerpt with paraphrase + citations"
- "Reduce excerpt size to within limits and add reuse_log"
- "Mark KID as reusable_with_allowlist and set allowed_excerpt (governed change)"

Piece E — Validation checklist (gate view)

File: `knowledge/contracts/KL-5.3_validation_checklist.md`

```
---
contract_id: KL-5.3d
schema_version: 1.0.0
applies_to: "KL reuse gate validation"
enforcement_level: "hard"
---

# KL-5.3d — Plagiarism / Reuse Gates Validation Checklist

- [ ] KL-GATE-06 requires reuse_log when verbatim excerpts exist
- [ ] KL-GATE-06 blocks reuse unless use_policy==reusable_with_allowlist
- [ ] KL-GATE-07 enforces per-KID excerpt limits (words/lines)
- [ ] Verbatim detection definition exists (>=12 words or >=3 lines or code match)
- [ ] Evidence format includes measurements + locations + remediation
```

KL-5.4 Quality gates (optional but strong)

KL-5.4 — Quality gates (optional but strong)

Piece A — Gate definition

File: `knowledge/contracts/KL-5.4_quality_gates.md`

```
---
contract_id: KL-5.4
schema_version: 1.0.0
applies_to: "Knowledge selection for higher-risk runs"
enforcement_level: "soft_to_hard_by_policy"
---

# KL-5.4 — Quality Gates

## KL-GATE-08 — Production maturity threshold for selected KIDs
**Rule:** For runs where risk_class in {PROD, COMPLIANCE}:
- Every selected KID MUST meet maturity >= required threshold.

Default thresholds:
- PROD: reviewed+
- COMPLIANCE: verified+

## Exceptions
- Exceptions are allowed ONLY if:
  - an override is explicitly recorded in the run (operator override), AND
  - the selection artifact notes the exception, AND
  - the output is tagged "knowledge_maturity_exception=true"

## Fail reason code
KL-GATE-08
```

Piece B — Machine-readable gate spec (for gate runner)

File: `knowledge/contracts/KL-5.4_quality_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "KL-5.4a",
  "gates": [
    {
```

```
    "gate_id": "KL-GATE-08",
    "name": "Production maturity threshold",
    "severity": "policy_controlled",
    "checks": [
      "if selection.inputs.risk_class==PROD then all(selected[].restrictions.maturity in
{reviewed,verified,golden})",
      "if selection.inputs.risk_class==COMPLIANCE then all(selected[].restrictions.maturity in
{verified,golden})",
      "if override_present then selection.notes include maturity_exception"
    ]
  }
 ]
}
```

Piece C — Override record format (for controlled exceptions)

File: `knowledge/contracts/KL-5.4_override_record.md`

```
---
contract_id: KL-5.4b
schema_version: 1.0.0
applies_to: "Operator override of KL-GATE-08"
enforcement_level: "hard"
---

# KL-5.4b — Override Record Format

If KL-GATE-08 is overridden, record:

- override_id: OVERRIDE-KL-GATE-08-<NNNN>
- operator: <name/id>
- timestamp: <iso datetime>
- reason: <why this is safe>
- impacted_kids: [KID-...]
- compensating_controls: <tests, audits, manual review, extra gates>
- expiry: <date> (must not be blank)

This record must be referenced by:
- run log
- KNOWLEDGE_SELECTION.notes
```

Piece D — Validation checklist (gate view)

File: `knowledge/contracts/KL-5.4_validation_checklist.md`

---
contract_id: KL-5.4c
schema_version: 1.0.0
applies_to: "KL quality gate validation"
enforcement_level: "hard_when_enabled"

KL-5.4c — Quality Gates Validation Checklist

- [ ] KL-GATE-08 exists with thresholds for PROD and COMPLIANCE
- [ ] Overrides require recorded operator + reason + expiry + compensating controls
- [ ] Overridden selections annotate the selection artifact notes

Say **“next”** for **KL-6.1 Growth model**.

KL-6) Scaling Over Time

KL-6) Scaling Over Time (content ops)

This is how it grows without rotting.

KL-6.1 Growth model

- add items atomically
- update taxonomy conservatively
- use bundles to create curated “slices”

KL-6.2 Maturity workflow

- draft → reviewed → verified → golden
- reviewed means: checked for correctness + policy alignment
- verified means: validated against real runs or tests
- golden means: stable canonical reference; changes are controlled

KL-6.3 Deprecation workflow

- mark deprecated
- add replacement KID
- keep aliases for a window
- block new selection of deprecated KIDs (except repro)

KL-6.4 External content ingestion rules

If you ingest outside material:

- store source metadata

- do not store copyrighted text verbatim unless licensed
- summarize + cite; keep excerpts within allowed limits
- enforce through `SOURCES_INDEX` + use_policy gates

KL-6.1 Growth model

KL-6.1 — Growth model

Piece A — Growth model contract (atomic additions, conservative taxonomy)

File: `knowledge/contracts/KL-6.1_growth_model.md`

```
---
contract_id: KL-6.1
schema_version: 1.0.0
applies_to: "Knowledge Library content operations"
enforcement_level: "hard"
---

# KL-6.1 — Growth Model

## Rule: Add items atomically
- New knowledge is added as **small KIDs** (no monolith docs).
- Each KID should cover a single concept/pattern/procedure/checklist/etc.
- If a topic needs more than ~1,200 words, split into multiple KIDs.

## Rule: Update taxonomy conservatively
- TAXONOMY changes are rare and governed:
  - Additions are allowed.
  - Renames/removals require DEPRECATIONS + CHANGELOG entry.
- Prefer tag_aliases to absorb naming drift.

## Rule: Use bundles for curated slices
- Instead of “select everything,” use BUNDLES_INDEX to create standard sets:
  - by run profile
  - by risk class
  - by executor type
  - by stack family

## Rule: Keep selection deterministic
- Selection depends only on:
  - selection inputs (KL-3.1)
  - registries (KL-2.x)
  - deterministic ranking rules (KL-3.3)

## Rule: Prevent rot
```

- Any KID touched should update updated_at and add a CHANGELOG note if behavior changes.
- Deprecate instead of overwriting when meaning changes materially.

Piece B — “Atomicity” checks (what counts as too big / too broad)

File: `knowledge/contracts/KL-6.1_atomicity_checks.md`

```
---
contract_id: KL-6.1a
schema_version: 1.0.0
applies_to: "KID authoring review"
enforcement_level: "soft_to_hard_by_policy"
---

# KL-6.1a — Atomicity Checks

A KID SHOULD be split if any are true:
- > 1,200 words (hard cap from KL-1.2)
- covers 3+ distinct subtopics that could stand alone
- has > 12 checklist items across multiple concerns
- includes both “what/why” and a deep “how-to” that could be separate concept + procedure

Split strategy:
- concept + pattern + procedure + checklist as separate KIDs
- keep cross-links in Links section
```

Piece C — Validation checklist (ops view)

File: `knowledge/contracts/KL-6.1_validation_checklist.md`

```
---
contract_id: KL-6.1b
schema_version: 1.0.0
applies_to: "KL growth model compliance review"
enforcement_level: "soft"
---

# KL-6.1b — Growth Model Validation Checklist
```

- [ ] New knowledge additions are atomic KIDs
- [ ] TAXONOMY changes are conservative and logged
- [ ] Bundles are used to create curated slices
- [ ] Selection remains deterministic (no ad-hoc roaming)
- [ ] Material meaning changes use deprecation rather than silent overwrite

Say **“next”** for **KL-6.2 Maturity workflow**.

KL-6.2 Maturity workflow

KL-6.2 — Maturity workflow

Piece A — Maturity workflow contract (draft → reviewed → verified → golden)

File: `knowledge/contracts/KL-6.2_maturity_workflow.md`

---
contract_id: KL-6.2
schema_version: 1.0.0
applies_to: "KID maturity transitions"
enforcement_level: "hard"

KL-6.2 — Maturity Workflow

States
- draft
- reviewed
- verified
- golden

Allowed transitions
- draft → reviewed → verified → golden

Reverse transitions are not allowed unless handled as:
- deprecate old KID
- create new KID
- map via DEPRECATIONS
- log in CHANGELOG

Definitions (what each maturity means)

draft
Meaning:
- authored but not formally checked

Minimum bar:
- passes KL-1.2 (sections) + KL-1.3 (metadata)
- indexed in KNOWLEDGE_INDEX

reviewed
Meaning:
- checked for correctness + policy alignment

Minimum bar:
- passes draft requirements
- reviewed by a human/operator checklist:
  - technically plausible
  - does not violate pattern-only default
  - correct use_policy / executor_access
- “Proof/confidence” evidence updated to include reviewer note

verified
Meaning:
- validated against real runs or tests

Minimum bar:
- passes reviewed requirements
- evidence includes at least one of:
  - referenced run_id where it was applied successfully
  - test case IDs that prove the guidance
  - audit log reference showing correct use
- any defaults/constraints match current standards snapshot

golden
Meaning:
- stable canonical reference; changes controlled

Minimum bar:
- passes verified requirements
- used across multiple runs without contradictions
- changes require:
  - CHANGELOG entry
  - if meaning changes materially: create new KID + deprecate old

---

Piece B — Review checklist template (for promoting to reviewed/verified/golden)

File: `knowledge/templates/KID_maturity_review_checklist.md`

KID Maturity Review Checklist

KID: <KID-...>
Current maturity: <draft|reviewed|verified|golden>
Target maturity: <reviewed|verified|golden>
Reviewer: <name/id>
Date: <YYYY-MM-DD>

Policy alignment
- [ ] use_policy is correct (pattern-only default preserved unless allowlisted)
- [ ] executor_access is correct (no internal-only leakage risk)
- [ ] allowed_excerpt is correct (0/0 unless explicitly allowlisted)

Content quality
- [ ] Summary is clear and short
- [ ] When to use is specific and matches tags/domains
- [ ] Do/Don't is concrete
- [ ] Core content matches type semantics (KL-1.1a)
- [ ] No monolith behavior (atomic)

Proof/confidence
- [ ] tier updated appropriately
- [ ] evidence added (for verified/golden)

Decision
- [ ] Approved
- [ ] Rejected (fixes required)
Notes:
- <notes>

Piece C — Validation checklist (gate/ops view)

File: `knowledge/contracts/KL-6.2_validation_checklist.md`

---
contract_id: KL-6.2a
schema_version: 1.0.0
applies_to: "Maturity workflow compliance"
enforcement_level: "soft_to_hard_by_policy"

KL-6.2a — Maturity Workflow Validation Checklist

- [ ] Maturity only progresses forward (draft→reviewed→verified→golden)
- [ ] reviewed requires reviewer record + policy alignment
- [ ] verified requires run/test/audit evidence
- [ ] golden changes are controlled and logged; meaning changes use deprecation

Say **“next”** for **KL-6.3 Deprecation workflow**.

KL-6.3 Deprecation workflow

KL-6.3 — Deprecation workflow

Piece A — Deprecation workflow contract

File: `knowledge/contracts/KL-6.3_deprecation_workflow.md`

```
---
contract_id: KL-6.3
schema_version: 1.0.0
applies_to: "Deprecating KIDs / bundles / taxonomy elements"
enforcement_level: "hard"
---

# KL-6.3 — Deprecation Workflow

## Goal
Retire outdated knowledge without breaking reproducibility.

## Workflow (KID)
1) Mark KID as deprecated
- Set frontmatter: deprecated_by: <replacement_kid>
- Update updated_at

2) Add DEPRECATIONS mapping
- Add record in DEPRECATIONS.json:
  - kind: kid
  - id: <deprecated_kid>
  - replacement_id: <replacement_kid>
  - reason + effective_date
  - allow_for_repro (usually true)

3) Update KNOWLEDGE_INDEX
- Ensure deprecated KID remains listed (do not delete immediately)
- Optional: add a field deprecated_by in the index entry (if you store it there)

4) Add CHANGELOG entry
- “Deprecated: <old> -> <new> (reason…)”

5) Enforce selection behavior
- Normal mode: deprecated excluded
- Repro mode: deprecated allowed only if allow_for_repro=true
```

Workflow (Bundle)
Same pattern:
- bundle.deprecated_by set
- DEPRECATIONS.kind=bundle mapping created
- CHANGELOG updated

Workflow (Tag/Domain/Subdomain)
1) Add deprecation mapping (DEPRECATIONS.kind=tag/domain/subdomain)
2) Add TAXONOMY alias mapping (old -> new) for migration window
3) Update CHANGELOG
4) Update impacted KIDs incrementally
5) After migration window: remove alias only when all KIDs are migrated (optional strict)

Piece B — Deprecation record template (copy/paste)

File: `knowledge/templates/DEPRECATION_record_template.json`

```
{
  "kind": "kid",
  "id": "KID-<...>",
  "replacement_id": "KID-<...>",
  "reason": "<why deprecated>",
  "effective_date": "2026-03-05",
  "allow_for_repro": true,
  "notes": "<migration notes>",
  "created_at": "2026-03-05",
  "owner": "axion"
}
```

Piece C — Migration window policy (how long aliases last)

File: `knowledge/contracts/KL-6.3_migration_window_policy.md`

---
contract_id: KL-6.3a
schema_version: 1.0.0
applies_to: "Taxonomy alias migration windows"
enforcement_level: "soft_to_hard_by_policy"

KL-6.3a — Migration Window Policy

Default migration window:
- 30 days OR 10 runs (whichever comes first)

During window:
- TAXONOMY.tag_aliases may map old -> new
- Selector normalizes to new tag

Exit criteria:
- No KID/index entry references the old tag/domain/subdomain

After window:
- old tag/domain may remain as alias (safe) OR be removed (strict),
  but removal requires:
  - changelog entry
  - verification that no references remain

Piece D — Validation checklist (ops view)

File: `knowledge/contracts/KL-6.3_validation_checklist.md`

---
contract_id: KL-6.3b
schema_version: 1.0.0
applies_to: "Deprecation workflow compliance"
enforcement_level: "hard"

KL-6.3b — Deprecation Workflow Validation Checklist

- [ ] Deprecated KIDs have deprecated_by set
- [ ] DEPRECATIONS.json contains mapping (kind + id -> replacement_id)
- [ ] CHANGELOG includes the deprecation
- [ ] Selection excludes deprecated in normal mode
- [ ] Repro mode allows deprecated only when allow_for_repro==true
- [ ] Taxonomy deprecations include alias migration window when needed

Say **“next”** for **KL-6.4 External content ingestion rules**.

KL-6.4 External content ingestion rules

KL-6.4 — External content ingestion rules

Piece A — External ingestion contract (no copyrighted dumps, summarize + cite)

File: `knowledge/contracts/KL-6.4_external_ingestion_rules.md`

```
---
contract_id: KL-6.4
schema_version: 1.0.0
applies_to: "Creating KIDs from non-internal sources"
enforcement_level: "hard"
---

# KL-6.4 — External Content Ingestion Rules

## Goal
Allow learning from external sources without turning the library into a copy-donor.

## Hard rules
1) No verbatim copyrighted text unless licensed for reuse
- If not clearly licensed for reuse, store only summaries/paraphrase.
- Any verbatim excerpt must be within:
  - source reuse_constraints (SOURCES_INDEX)
  - KID.allowed_excerpt
  - and must be recorded in reuse_log if used in outputs

2) SOURCES_INDEX governs provenance
If a KID is derived from external material:
- it MUST reference a source_id from SOURCES_INDEX
- source_id must define license + allowed_reuse_level

3) Prefer summary-first
Default external ingestion output should be:
- concept/pattern/procedure/checklist written in your own words
- minimal quotes/excerpts (ideally none)

4) Mandatory attribution when required
If SOURCES_INDEX.attribution_required == true:
- Links section must include attribution (publisher/author/year or equivalent)

## Allowed storage modes (based on allowed_reuse_level)
```

- summary_only: store only paraphrase + citations
- short_excerpts_only: store small excerpts, still bounded
- allowlisted_reuse: store allowlisted fragments only (explicit)
- full_reuse_per_license: still cite; keep atomic

Export safety
External kit exports must obey:
- executor_access rules
- use_policy rules
- excerpt limits
Even if a source license permits reuse, your library policy may still restrict it.

Piece B — “External-derived KID” metadata extension (optional field)

File: `knowledge/contracts/KL-6.4_external_metadata_extension.md`

---
contract_id: KL-6.4a
schema_version: 1.0.0
applies_to: "KID metadata when derived from external content"
enforcement_level: "soft_to_hard_by_policy"

KL-6.4a — External Metadata Extension

If a KID is derived from an external source, add these optional metadata fields:

- source_id: "SRC-..."
- source_attribution: "Publisher / Author / Year"
- derivation_mode: summary_only | short_excerpts_only | allowlisted_reuse | full_reuse_per_license

These fields improve auditing but are optional if you keep attribution in Links.

Piece C — Ingestion checklist (operator/author workflow)

File: `knowledge/templates/EXTERNAL_INGESTION_checklist.md`

External Ingestion Checklist

Source:
- source_id: <SRC-...>
- license: <...>
- allowed_reuse_level: <...>
- attribution_required: <true|false>

Before writing
- [ ] Source exists in SOURCES_INDEX
- [ ] Reuse constraints understood (max excerpt words/lines)
- [ ] Decide derivation_mode (summary_only recommended)

Writing the KID
- [ ] Use your own words (summary-first)
- [ ] No large verbatim blocks
- [ ] If excerpts included, ensure allowed_excerpt is set and policy is allowlisted
- [ ] Add attribution in Links if required

Indexing
- [ ] Add/verify KNOWLEDGE_INDEX entry
- [ ] Ensure tags/domains are canonical (TAXONOMY)

Export safety
- [ ] Confirm executor_access + use_policy are correct

Piece D — Validation checklist (gate/ops view)

File: `knowledge/contracts/KL-6.4_validation_checklist.md`

---
contract_id: KL-6.4b
schema_version: 1.0.0
applies_to: "External ingestion compliance validation"
enforcement_level: "hard"

KL-6.4b — External Ingestion Validation Checklist

- [ ] If KID derived from external source:
  - [ ] source_id exists in SOURCES_INDEX (strict mode)
  - [ ] license + allowed_reuse_level present
  - [ ] attribution included when required
- [ ] No verbatim copyrighted dumps outside allowed reuse constraints

- [ ] Any verbatim excerpt intended for outputs is:
  - [ ] allowlisted (use_policy == reusable_with_allowlist)
  - [ ] within KID.allowed_excerpt
  - [ ] within source reuse_constraints

Say **“next”** for **KL-7 Minimum Viable Knowledge Library**.

KL-7) Minimum Viable Knowledge Library

KL-7) Minimum Viable Knowledge Library (what you should build first)

Given your “everything IT + industries + languages” goal, don’t start with “everything.” Start with a scalable spine:

1. **Taxonomy + index system**
2. **IT_END_TO_END**: security, networking, databases, auth, CI/CD, observability (core)
3. **Languages**: TypeScript/Node, React, SQL/Postgres (core)
4. **Industry playbooks**: 3–5 industries to start (healthcare, finance, ecommerce, logistics, gov)

KL-7 — Minimum Viable Knowledge Library (MVKL)

KL-7 — Minimum Viable Knowledge Library (MVKL)

Piece A — MVKL build order (the “spine”)

File: `knowledge/contracts/KL-7_mvkl_build_order.md`

```
---
contract_id: KL-7
schema_version: 1.0.0
applies_to: "Initial Knowledge Library rollout plan"
enforcement_level: "soft"
---

# KL-7 — Minimum Viable Knowledge Library

## Principle
Do not start with “everything.” Start with a scalable spine:
- registries + selection + gates first
- then a small, high-leverage content set
- then expand via atomic KIDs + bundles

## Phase 0 — System spine (must exist first)
Create and validate:
- KL-1 (KID model)
- KL-2 (registries)
- KL-3 (selection + KNOWLEDGE_SELECTION)
- KL-4 (usage rules + citations)
- KL-5 (gates)
- KL-6 (ops workflows)

## Phase 1 — Core IT_END_TO_END domains (minimum)
Add these domains as first-class in TAXONOMY and content:
1) security_fundamentals
2) auth_identity
3) databases_postgres (or databases_core)
4) networking_http
5) cicd_release
6) observability_core

Each domain should start with:
- 2 concepts
- 2 patterns
```

- 1 procedure
- 1 checklist
- 1 pitfall
- 1 reference

Total per domain: ~8 KIDs

Phase 2 — Core languages/stacks (minimum)

Add:
- TypeScript/Node (backend)
- React (frontend)
- SQL/Postgres (data)

Each starts with:
- 1 reference (defaults/enums)
- 2 patterns
- 1 checklist
- 1 pitfall

Phase 3 — Industry playbooks (start with 3–5)

Recommended starting set:
- healthcare
- finance
- ecommerce
- logistics
- gov

Each industry starts with:
- 1 concept (risk landscape)
- 1 checklist (compliance basics)
- 1 pitfall (common failures)
- 1 reference (required artifacts / documentation norms)

Phase 4 — Bundles (make selection usable)

Create bundles:
- by run_profile: API / WEB / MOBILE
- by risk_class: PROTOTYPE / PROD / COMPLIANCE
- by executor: internal / external
- by stack_family: react / node / postgres

Exit criteria for “MVKL complete”

- Selection produces stable KNOWLEDGE_SELECTION artifacts
- Gates block policy violations
- At least 2 bundles exist (API-PROTOTYPE, API-PROD)
- At least 1 IT_END_TO_END domain has a complete starter set (~8 KIDs)

Piece B — MVKL “starter KID set” template (per domain)

File: `knowledge/templates/MVKL_domain_starter_set.md`

```
# MVKL Domain Starter Set Template

Domain: <domain_slug>
Pillar: <IT_END_TO_END | ...>

## Required KIDs (minimum)
- Concepts (2)
 - KID-...-CONCEPT-0001: <title>
 - KID-...-CONCEPT-0002: <title>

- Patterns (2)
 - KID-...-PATTERN-0001: <title>
 - KID-...-PATTERN-0002: <title>

- Procedure (1)
 - KID-...-PROCEDURE-0001: <title>

- Checklist (1)
 - KID-...-CHECKLIST-0001: <title>

- Pitfall (1)
 - KID-...-PITFALL-0001: <title>

- Reference (1)
 - KID-...-REFERENCE-0001: <title>

## Bundle suggestion
- BUNDLE-<DOMAIN>_STARTER-0001 includes all above KIDs (ordered)
```

Piece C — Validation checklist (MVKL readiness)

File: `knowledge/contracts/KL-7_validation_checklist.md`

```
---
contract_id: KL-7a
schema_version: 1.0.0
applies_to: "MVKL readiness review"
```

enforcement_level: "soft"

KL-7a — MVKL Validation Checklist

Spine
- [ ] KL-1 through KL-6 artifacts exist
- [ ] Registries validate (KL-2.x)
- [ ] Selection input/output validate (KL-3.1, KL-3.2)
- [ ] Usage rules + citations enforced (KL-4.x)
- [ ] Gates exist and are runnable (KL-5.x)

Content minimum
- [ ] At least 1 IT_END_TO_END domain has ~8 KIDs
- [ ] KNOWLEDGE_INDEX references all KIDs with correct paths
- [ ] TAXONOMY contains the domain/tags used
- [ ] At least 2 bundles exist and validate

Safety
- [ ] External executor selection and export rules are enforced
- [ ] Reuse gates block verbatim beyond limits

IT_END_TO_END Pillar

IT_END_TO_END

01_foundations

- `security_fundamentals` — Security Fundamentals
- `identity_and_access_management` — Identity & Access Management (IAM)
- `application_security` — Application Security (AppSec)
- `cryptography_and_key_management` — Cryptography & Key Management
- `networking_fundamentals` — Networking Fundamentals
- `internet_protocols` — Internet Protocols (TCP/IP, UDP, DNS)
- `http_and_web_platform` — HTTP & Web Platform Fundamentals
- `operating_systems` — Operating Systems (Linux/Windows)
- `shell_and_cli` — Shell & CLI Fundamentals
- `data_structures_and_algorithms` — Data Structures & Algorithms
- `software_architecture` — Software Architecture
- `distributed_systems` — Distributed Systems Fundamentals

02_data_and_storage

- `relational_databases` — Relational Databases (RDBMS)
- `postgresql` — PostgreSQL
- `mysql_mariadb` — MySQL / MariaDB
- `sql_modeling_and_migrations` — SQL Modeling & Migrations

- `nosql_datastores` — NoSQL Datastores
- `document_databases` — Document Databases (MongoDB etc.)
- `key_value_stores` — Key-Value Stores (Redis etc.)
- `search_and_indexing` — Search & Indexing (Elasticsearch/OpenSearch)
- `data_warehousing` — Data Warehousing
- `data_lakes_and_lakehouse` — Data Lakes / Lakehouse
- `object_storage` — Object Storage (S3-compatible)
- `backup_restore_and_dr` — Backup, Restore & Disaster Recovery

03_backend_and_apis

- `api_design` — API Design
- `rest_apis` — REST APIs
- `graphql` — GraphQL
- `grpc_and_rpc` — gRPC / RPC
- `webhooks_and_event_delivery` — Webhooks & Event Delivery
- `authentication_and_sessions` — Authentication & Sessions
- `authorization_models` — Authorization Models (RBAC/ABAC/ReBAC)
- `rate_limiting_and_abuse_controls` — Rate Limiting & Abuse Controls
- `input_validation_and_sanitization` — Input Validation & Sanitization
- `background_jobs_and_queues` — Background Jobs & Queues
- `caching_strategies` — Caching Strategies

- `feature_flags_and_config` — Feature Flags & Configuration
- `api_observability` — API Observability

04_frontend_and_clients

- `web_frontend_engineering` — Web Frontend Engineering
- `mobile_engineering` — Mobile Engineering
- `desktop_applications` — Desktop Applications
- `ui_architecture` — UI Architecture (State, Routing, Data Fetching)
- `design_systems` — Design Systems
- `accessibility` — Accessibility (a11y)
- `performance_frontend` — Frontend Performance
- `security_frontend` — Frontend Security (XSS/CSRF/CSP)
- `offline_and_sync` — Offline & Sync
- `push_notifications` — Push Notifications
- `internationalization` — Internationalization (i18n)

05_infrastructure_and_platform

- `cloud_fundamentals` — Cloud Fundamentals
- `containers` — Containers (Docker)
- `kubernetes` — Kubernetes
- `service_mesh` — Service Mesh
- `infrastructure_as_code` — Infrastructure as Code (IaC)

- `networking_cloud` — Cloud Networking
- `load_balancing_and_edge` — Load Balancing & Edge/CDN
- `secrets_and_config_platform` — Secrets & Config (Platform)
- `observability_platform` — Observability (Platform)
- `platform_engineering` — Platform Engineering
- `internal_developer_platforms` — Internal Developer Platforms (IDPs)

06_devops_and_reliability

- `ci_cd` — CI/CD
- `release_engineering` — Release Engineering
- `site_reliability_engineering` — Site Reliability Engineering (SRE)
- `incident_response` — Incident Response
- `monitoring_and_alerting` — Monitoring & Alerting
- `logging_and_tracing` — Logging & Tracing
- `slo_sla_error_budgets` — SLO/SLA & Error Budgets
- `capacity_planning` — Capacity Planning
- `performance_and_scalability` — Performance & Scalability
- `cost_optimization` — Cost Optimization (FinOps)
- `quality_engineering` — Quality Engineering (QE)
- `testing_strategies` — Testing Strategies

07_data_engineering_and_ai

- `data_engineering` — Data Engineering
- `etl_elt_pipelines` — ETL/ELT Pipelines
- `stream_processing` — Stream Processing
- `event_streaming` — Event Streaming (Kafka/Pulsar)
- `analytics_and_telemetry` — Analytics & Telemetry
- `ml_engineering` — ML Engineering
- `mlops` — MLOps
- `llm_applications` — LLM Applications
- `retrieval_and_rag` — Retrieval & RAG
- `vector_databases` — Vector Databases
- `model_serving` — Model Serving
- `ai_safety_and_governance` — AI Safety & Governance

08_security_operations_and_compliance

- `security_operations` — Security Operations (SecOps)
- `threat_detection_and_response` — Threat Detection & Response
- `vulnerability_management` — Vulnerability Management
- `penetration_testing` — Penetration Testing
- `secure_sdls` — Secure SDLC
- `compliance_frameworks` — Compliance Frameworks (SOC2/ISO/HIPAA/PCI)
- `privacy_and_data_protection` — Privacy & Data Protection

- `governance_risk_and_compliance` — GRC (Governance, Risk & Compliance)
- `audit_and_logging` — Audit & Logging

LANGUAGES_AND_LIBRARIES Pillar

LANGUAGES_AND_LIBRARIES

01_programming_languages

- `javascript` — JavaScript
- `typescript` — TypeScript
- `python` — Python
- `go` — Go
- `rust` — Rust
- `java` — Java
- `csharp_dotnet` — C# / .NET
- `cpp` — C++
- `c` — C
- `swift` — Swift
- `kotlin` — Kotlin
- `solidity` — Solidity
- `move` — Move (Aptos/Sui ecosystems)
- `vyper` — Vyper
- `sql` — SQL
- `bash_shell` — Bash / Shell Scripting

02_web_frameworks

- `react` — React
- `nextjs` — Next.js
- `vue` — Vue
- `svelte` — Svelte
- `angular` — Angular
- `nodejs` — Node.js
- `express` — Express
- `nestjs` — NestJS
- `fastapi` — FastAPI
- `django` — Django
- `flask` — Flask
- `spring_boot` — Spring Boot

03_mobile_frameworks

- `react_native` — React Native
- `expo` — Expo
- `flutter` — Flutter
- `ios_swiftui` — iOS (SwiftUI/UIKit)
- `android_kotlin` — Android (Kotlin/Jetpack)

04_databases_and_orms

- `drizzle_orm` — Drizzle ORM

- `prisma` — Prisma
- `typeorm` — TypeORM
- `sequelize` — Sequelize
- `sqlalchemy` — SQLAlchemy
- `postgres_drivers` — PostgreSQL Drivers
- `redis_clients` — Redis Clients

05_cloud_and_devops_tooling

- `docker` — Docker
- `kubernetes_tools` — Kubernetes Tooling
- `terraform` — Terraform
- `pulumi` — Pulumi
- `ansible` — Ansible
- `github_actions` — GitHub Actions
- `gitlab_ci` — GitLab CI
- `circleci` — CircleCI
- `cloudflare_platform` — Cloudflare Platform
- `aws` — Amazon Web Services (AWS)
- `gcp` — Google Cloud Platform (GCP)
- `azure` — Microsoft Azure

06_blockchain_stacks

- `ethereum` — Ethereum
- `polygon` — Polygon
- `solana` — Solana
- `cosmos_sdk` — Cosmos SDK
- `substrate_polkadot` — Substrate / Polkadot
- `chainlink` — Chainlink
- `ipfs` — IPFS
- `walletconnect` — WalletConnect
- `wagmi_rainbowkit` — Wagmi / RainbowKit
- `hardhat_foundry` — Hardhat / Foundry
- `openzeppelin` — OpenZeppelin

07_audio_music_tech

- `daw_concepts` — Digital Audio Workstations (DAW) Concepts
- `midi` — MIDI
- `audio_plugins_vst_au_aax` — Audio Plugins (VST/AU/AAX)
- `dsp_audio` — Digital Signal Processing (DSP) for Audio
- `audio_routing_and_latency` — Audio Routing & Latency
- `pro_tools` — Avid Pro Tools
- `ableton_live` — Ableton Live
- `logic_pro` — Logic Pro

- `fl_studio` — FL Studio
- `reaper` — REAPER
- `studio_one` — Studio One

08_3d_and_graphics_tech

- `blender` — Blender
- `maya` — Autodesk Maya
- `3ds_max` — Autodesk 3ds Max
- `cinema_4d` — Cinema 4D
- `houdini` — Houdini
- `unreal_engine` — Unreal Engine
- `unity` — Unity
- `substance_3d` — Adobe Substance 3D
- `pbr_materials` — Physically Based Rendering (PBR) Materials
- `rendering_pipelines` — Rendering Pipelines
- `rigging_and_animation` — Rigging & Animation
- `photogrammetry` — Photogrammetry

09_video_streaming_and_realtime

- `webrtc` — WebRTC
- `rtmp` — RTMP
- `hls_dash` — HLS / MPEG-DASH

- `ffmpeg` — FFmpeg
- `streaming_architecture` — Streaming Architecture
- `realtime_messaging` — Realtime Messaging (WebSockets)
- `signaling_systems` — Signaling Systems
- `media_storage_delivery` — Media Storage & Delivery

INDUSTRY_PLAYBOOKS Pillar

INDUSTRY_PLAYBOOKS

01_regulated_industries

- `healthcare` — Healthcare (HIPAA)
- `financial_services` — Financial Services
- `payments` — Payments (PCI DSS)
- `insurance` — Insurance
- `government_public_sector` — Government / Public Sector
- `education` — Education (FERPA)
- `legal_services` — Legal Services
- `energy_utilities` — Energy & Utilities

02_commerce_and_operations

- `ecommerce` — E-commerce
- `retail` — Retail
- `logistics_supply_chain` — Logistics & Supply Chain
- `manufacturing` — Manufacturing
- `construction` — Construction
- `real_estate` — Real Estate
- `hospitality_travel` — Hospitality & Travel
- `food_and_beverage` — Food & Beverage

03_media_and_creator_economy

- `music_and_audio` — Music & Audio
- `film_and_video` — Film & Video
- `gaming` — Gaming
- `livestreaming_platforms` — Livestreaming Platforms
- `digital_publishing` — Digital Publishing
- `creator_tools` — Creator Tools

04_emerging_tech_industries

- `blockchain_web3` — Blockchain / Web3
- `ai_products` — AI Products
- `iot` — Internet of Things (IoT)
- `cybersecurity_products` — Cybersecurity Products
- `devtools` — Developer Tools
- `saas_b2b` — B2B SaaS
- `marketplaces` — Marketplaces
- `social_platforms` — Social Platforms / Communities